

```

start = end = tempCookie.indexOf("["); //finds first instance of the right bracket

for(var i=0; i<3; i++) //finds the end of the center lat/long/zoom level
end = tempCookie.indexOf("]",end)+1;

temp = tempCookie.substring(start,end); //stores the center information

parseStart = parseEnd = tempCookie.indexOf("(" + info, tempCookie.indexOf("DELIMITER")+9); //finds the
indexes of the marker elements
for(var i=0; i<5; i++)
parseEnd=tempCookie.indexOf(")",parseEnd)+1;

markerCookieHalf1 = tempCookie.substring(tempCookie.indexOf("DELIMITER")+9, parseStart); //portion before the
marker information to delete
markerCookieHalf2 = tempCookie.substring(parseEnd, tempCookie.length); //portion
after the marker information to delete

setCookie(temp+"DELIMITER"+markerCookieHalf1+markerCookieHalf2); //rewrites
the cookie to include the new information

if(deletePoly == "true")
deletePolyLines(info, temp); //deletes polyline information

countCookies(); //updates the number of items
}

function deletePolyLines(info, startLocation) //deletes all polylines that contain the markerID of the marker
being deleted
{
    var polyDefStart = 0; //start of the polyline definition
    var polyDefEnd = 0; //end of the polyline definition
    var CookieHalf1, CookieHalf2; //For remainder of the cookie
    var otherIDStart, otherIDEnd; //beginning and ending for the other id that makes up the
    polyline //holds the other ID
    var otherID; //holds polyline's index
    var index; //stores the cookie
    var tempCookie;

    while(polyDefStart >= 0 && polyDefEnd >= 0) //while not yet reached the end of the cookie data
    {

```

```

tempCookie = getCookie();

polyDefStart = tempCookie.indexOf("{"+info, 0); //look for the marker's ID
polyDefEnd = tempCookie.indexOf(info+"", 0); //finds id and end bracket

if(polyDefEnd >= 0)
{
    var myString = info+"";
    give proper index
    polyDefEnd += myString.length;

}

if(polyDefStart > 0)
{
    polyDefEnd = tempCookie.indexOf("}", polyDefStart);
    otherIDStart = tempCookie.indexOf(" ", polyDefStart)+1;
    otherIDEnd = polyDefEnd;
    polyDefEnd += 1;
    otherID = tempCookie.substring(otherIDStart, otherIDEnd);
    index = info+otherID;
}

clicked first
//if the ID is the first ID in the expression

else if (polyDefEnd > 0)
{
    polyDefStart = searchBackwards(polyDefEnd, "{");
    otherIDStart = polyDefStart+1;
    otherIDEnd = tempCookie.indexOf(" ", polyDefStart);
    otherID = tempCookie.substring(otherIDStart, otherIDEnd);
    index = otherID+info;
}

clicked first
//concatenates the two ids for the polyline index
//their order depends on which was

else
break;
//stop the loop

map.removeOverlay(polyline[index]);
//removes the polyline from the map

CookieHalf1 = tempCookie.substring(tempCookie.indexOf("DELIMITER")+9, polyDefStart);
//portion before the
polyline information to delete

```

CookieHalf2 = tempCookie.substring(polyDefEnd, tempCookie.length);
after the polyline information to delete

//portion

setCookie(startLocation+"DELIMITER"+CookieHalf1+CookieHalf2);
cookie

//sets the

polyDefStart = 0;
polyDefEnd = 0;

//resets polyDefStart and polyDefEnd

}

}

function searchBackwards(index, searchItem) //used when the ID being looked for is the second element in the
polyline definition to go back and get the location of the first element

{
var i;
var tempCookie = getCookie(); //gets the whole cookie

for(i=1; index-i>=0; i++) //starting at the index given, works backwards to
search for the element given
if(tempCookie.substring(index-i, index-i+1) == searchItem)
return index - i; //if item matches, then return the index where it is found

}

function changeComments(markerID) //function to change the comments of existing markers

{
var string = []; //holds properties from cookie

string = getMarkerInfo(markerID); //gets the markers information from the cookie

lat = string[0]; //stores marker property values into respective

variables

long = string[1];
comment = string[2];
tempColor = string[3];

deleteOneMarker(markerID, "false"); //calls the function to delete the marker, don't remove polyline

comment = prompt('Please enter a new comment:', ""); //prompts the user for a new comment to be added
if(comment == null)
show any value //if the comment is null, don't

```

comment = "";

icon.image = "http://labs.google.com/ridefinder/images/mm_20_" + tempColor + ".png"; //sets the icon global
variable temporarily

updateCookie("(" + markerID + ") (" + lat + ") (" + long + ") (" + comment + ") (" + tempColor + ")"); //stores new info in
the cookie

coord = new GLatlng(lat, long); //makes the new coordinate
object

createMarker(coord, comment, markerID); //calls the create marker function

icon.image = "http://labs.google.com/ridefinder/images/mm_20_" + color + ".png"; //the icon is set back to the
original color

}

function getMarkerInfo(markerID) //given a marker ID, this function returns the values of a marker in an
array
{
    var lat, long, comment="", tempColor; //stores marker's properties
    var tempCookie = getCookie(); //stores cookie
    var startIndex, endIndex; //start and end indexes for marker properties
    var string = []; //holds properties from cookie
    var coord; //holds marker's coordinates

    startIndex = tempCookie.indexOf("(" + tempCookie.indexOf(markerID + ")") + 1); //finds the index of the first
marker element
    endIndex = tempCookie.indexOf(")", startIndex); //finds the closing
    parenthesis

    for(var i=0; i<4; i++) //loops through to get all data elements
    {
        string[i] = tempCookie.substring(startIndex, endIndex);
        startIndex = endIndex+2;
        endIndex = tempCookie.indexOf(")", startIndex);
    }

    return string;
}

```

```

function createRecord(location, message) //creates a record, and calls create marker to create the marker
{
    var start, end; //stores the start and end of the latitude and longitude
    var first, second; //stores the latitude and longitude
    var coord; //stores the lat/long object google maps uses
    var info = ""; //stores comment info

    var marker; //stores marker object

    if(message != "") //if there was an included message
        info = message;
    else
        if(document.makeCheckBox.comments.checked)
        {
            info=prompt('Enter marker comment:', ""); //prompts the user for a marker comment
            if(info == null)
                info = "";
        }
    }

    start = location.indexOf("(") + 1; //gets the start position of the latitude
    end = location.indexOf(","); //gets the end position of the latitude
    first = location.substring(start, end); //stores the latitude

    start = location.indexOf(", ", end) + 2; //gets the start position of the longitude
    end = location.indexOf(")", start); //gets the end position of the longitude
    second = location.substring(start, end); //stores the longitude

    coord = new GLatLng(first, second); //makes a coordinate object
    marker = createMarker(coord, info, ""); //creates a marker

    updateCookie("(" + marker.info + ") (" + first + //sends the entire string to be stored in the cookie,
    "(" + second + ") (" + info + ") (" + color + ")"); //current marker color. marker.info stores creation time

    return marker;
}

function createMarker(point, message, info) //creates markers with lat/long and marker's message and unique
    identifier
{
    var marker = new GMarker(point, icon); //new marker
    if(info == "") //if there is no previous information

```

```

    {
        info = new Date();
        marker.info = info.getTime();
    }
    //date in milliseconds, used as a unique identifier
    else
    {
        marker.info = info;
        //sets marker info from the info from the
    }
    cookie

    map.addOverlay(marker);
    //adds marker to map

    GEvent.addListener(marker, "click", function() //calls click listener function
    {
        markerClickListener(marker, message); //calls markerClickListener function
    });

    return marker;
    //returns the marker
}

function markerClickListener(marker, message) //function used to create the marker's listener
{
    //as long as left click is not being used for any other purpose, show pop up window
    if(!document.deleteMarkerCheckbox.deleteMarker.checked &&
        !document.changeCommentCheckbox.changeComment.checked &&
        !document.makeCheckbox.make.checked &&
        !document.makePolyLine.makePoly.checked)
    {
        marker.openInfoWindowHtml("<big>" + message + "</big>");
    }
}

function deleteMarkers()
{
    //this clears all markers from the map and removes the data from the
    cookie
    {
        var tempCookie = getCookie();
        var start, end, temp;

        start = end = tempCookie.indexOf("[");
        //finds first instance of the right bracket

        for(var i=0; i<3; i++)
        {
            end = tempCookie.indexOf("]", end)+1;
            //finds the end of the center lat/long/zoom level
        }

        temp = tempCookie.substring(start, end); //stores the center information

        var exdate = new Date();

```

```

    exdate.setDate(exdate.getDate()+400);    //cookie expires 400 days in the future

    document.cookie = c_name + "=" + temp + "DELIMITER" + ";" + ";expires="+exdate;    //rewrites the cookie
    map.clearOverlays();
    with location.reload(true);
}

function makeNewPoly(markerID, newMarker) //makes a new polyline between two markers
{
    if(currentMarkerID == 0)                //a line must have two points
    {
        currentMarkerID = markerID;        //currentMarkerID is a global variable that stores the first
        marker's ID
    }
    else if(currentMarkerID == markerID)
    {
        //do nothing ;
    }
    else
    {
        var markerID1 = currentMarkerID;    //ID of first marker
        var markerID2 = markerID;           //ID of second marker

        var string1 = [];                  //string array to hold first markers properties
        var string2 = [];                  //string array to hold second markers properties
        var points = [];                  //holds polyline's points
        var lineColor;                    //holds color of first marker to decide what color to make the

        polyline

        var red = "#ff0000";
        var blue = "#0000ff";
        var black = "#000000";
        var green = "#00ff00";
        var brown = "#a37500";
        var white = "#ffffff";
        var yellow = "#ffff00";
        var orange = "#eebb00";
        var gray = "#aaaaaa";
        var purple = "#bf00be";
    }
}

```

```

string1 = getMarkerInfo(markerID1);           //gets first marker's info
string2 = getMarkerInfo(markerID2);           //gets second marker's info

points[0] = new GLatLng(string1[0], string1[1]); //string#[0] == latitude
points[1] = new GLatLng(string2[0], string2[1]); //string#[1] == longitude

lineColor = string1[3];                        //color to use for polyline

var mstring1;
var mstring2;                                //used to concatenate the two IDs into one

integer

mstring1 = markerID1.toString();              //convert ints to strings
mstring2 = markerID2.toString();

if(newMarker == "true")                       //if this is a new marker, create object and store in
{
    storePoly(mstring1, mstring2);
}

switch(lineColor)
{
    case "red": polyline[mstring1+mstring2] = new GPolyline(points, red,3,0.7); //creates
        polyline, stores in an array/dictionary
        break;
    case "blue": polyline[mstring1+mstring2] = new GPolyline(points, blue,3,0.7); //creates
        polyline, stores in an array/dictionary
        break;
    case "black": polyline[mstring1+mstring2] = new GPolyline(points, black,3,0.7); //creates
        polyline, stores in an array/dictionary
        break;
    case "green": polyline[mstring1+mstring2] = new GPolyline(points, green,3,0.7); //creates
        polyline, stores in an array/dictionary
        break;
    case "brown": polyline[mstring1+mstring2] = new GPolyline(points, brown,3,0.7); //creates
        polyline, stores in an array/dictionary
        break;
    case "white": polyline[mstring1+mstring2] = new GPolyline(points, white,3,0.7); //creates
        polyline, stores in an array/dictionary
        break;
    case "yellow": polyline[mstring1+mstring2] = new GPolyline(points, yellow,3,0.7); //creates
        polyline, stores in an array/dictionary

```



```

        break;
        case "orange": polyline[mstring1+mstring2] = new GPolyline(points, orange,3,0.7); //creates
        polyline, stores in an array/dictionary //creates
        break;
        case "gray": polyline[mstring1+mstring2] = new GPolyline(points, gray,3,0.7);
        polyline, stores in an array/dictionary //creates
        break;
        case "purple": polyline[mstring1+mstring2] = new GPolyline(points, purple,3,0.7);
        polyline, stores in an array/dictionary //creates
        break;
    }
    map.addOverlay(polyline[mstring1+mstring2]); //adds polyline to the map
    currentMarkerID = 0; //resets currentMarkerID
}

function storePoly(ID1, ID2) //stores the polyline data in the cookie
{
    var wholeString = getCookie(); //gets the cookie
    var sIndex;
    for(var i=0; i<items; i++)
    {
        sIndex = wholeString.indexOf(" ", parseStringStart) + 1;
    }
    updateCookie(""+ID1+", "+ID2+"");
}

function disableComment() //determines which checkboxes should be disabled and which should be disabled
{
    if(document.makeCheckBox.checked) //if the makeComment checkbox is
    checked: //enables the comment checkbox
    {
        document.makeCheckBox.comments.disabled = false;
    }
}

```

```

else if (document.deleteMarkerCheckBox.checked) //if the delete marker checkbox is checked:
{
    //if not the make comment

```

```
document.changeCommentCheckBox.changeComment.disabled = true; //disables the change comment checkbox
document.makePolyLine.makePoly.disabled = true; //disables the make polyline
```

```

    else if (document.changeCommentCheckBox.checked) //if the change comment checkbox is checked:
    {
        document.makeCheckBox.make.disabled = true; //disables the make comment
    }

```

```
document.deleteMarkerCheckBox.deleteMarker.disabled = true; //disables delete marker checkbox
document.makePolyLine.makePoly.disabled = true; //disables the make polyline
```

```
else if(document.makePolyLine.makePoly.checked)
{
    //if the make polyline checkbox is
}
```

```
document.makeCheckBox.make.disabled = true;
//disables the make comment
```

```
document.changeCommentCheckBox.disabled = true;
//disables the change comment checkbox
document.deleteMarkerCheckBox.disabled = true;
//disables delete marker checkbox
```

```
///if none of the above checkboxes are checked:
```

```

else
{
    //if none of the above checkboxes are checked
    //...then the comment checkbox

```

```
document.makeCheckBox.disabled = false; //enables the make comment checkbox
document.changeCommentCheckBox.changeComment.disabled = false; //enables the change comment checkbox
document.makePolyLine.makePoly.disabled = false; //enables the make polyline
```

checkbox

```

    }

    function iconColor(changedColor)    //changes the color of the icons used by the program
    {
        color = changedColor;          //changes current color to the value selected

        document.getElementById("colorDropDownMenu2").value = color;    //drop down menu in form set to current
        document.colorOptions.colorDropDownMenu.value = color;          //main drop down menu set to
        current color

        icon.image = "http://labs.google.com/ridefinder/images/mm_20_"+color+".png"; //calls the right icon from
        google's server
    }

    function showAddress(address)    //function for the geocode search bar displayed on top
    {
        if (geocoder)                //if the geocoder has been created
        {
            geocoder.getLatLng(address, function(point) {
                if (!point)
                    alert(address + " not found");    //if the address cannot be found
                else
                    map.setCenter(point, 13);          //if the address can be found
            });
        }
    }

    function showCookie()    //used for debugging purposes, displays the contents of the cookie
    {
        var temp = document.cookie;
        var newTemp = " ";
        var i;

        newTemp += temp.substring(0,100)+" ";

        for(i=200; i<temp.length; i+=100)
        {
            newTemp += temp.substring(i-100,i) + " ";
        }
    }

```

```

newTemp += temp.substring(i-100, temp.length);
alert(newTemp);
}
//]]>
</script>
</head>
<body onload="init()" onunload="GUNload()">
<table border="0" width="100%" cellpadding="0">
<td width="43%" valign="top">
<form name="geocode" action="#" onsubmit="showAddress(this.address.value); return false">
  <input type="text" size="60" name="address" value="" />
  <input type="submit" value="Go to New Location" />
  <!--<b>jumps to a location.</b>-->
</form>
</td>
<td>
<form>
<p>
<br>
<form name = "colorOptions">
<input type="button" value="Store Session Center"
  onclick="storeSessionCenter()">
<b>&nbsp;&nbsp; Store center and zoom level.&nbsp;&nbsp; </b>
<select name="colorDropDownMenu" onChange="iconColor(colorDropDownMenu.options[selectedIndex].value)">
<option value="red">Red
<option value="orange">Orange
<option value="brown">Brown

```

```

<option value="yellow">Yellow
<option value="green">Green
<option value="blue">Blue
<option value="purple">Purple
<option value="black">Black
<option value="gray">Gray
<option value="white">White
</select>
<b>&nbsp;Marker Color</b>

</form>

</td>

<td width="26%" valign="top">
<b>
  <form name="makeCheckBox">
    <input type="checkbox" name="make" onchange="disableComment()">Make a marker on click
    <input type="checkbox" name="comments">with a comment.
  </form>
</b>

<b>
  <form name="changeCommentCheckBox">
    <input type="checkbox" name="changeComment" onchange="disableComment()">Change an existing marker's comment.
  </form>
</b>

<b>
  <form name="deleteMarkerCheckBox">
    <input type="checkbox" name="deleteMarker" onchange="disableComment()">Delete a marker when it is clicked.
  </form>
</b>

<br>

<b>
  <form name="makePolyline">
    <input type="checkbox" name="makePoly" onchange="disableComment()">Make a polyline between two markers.
  </form>
</b>

<b>

```

```

<form name="hideMap">
  <input type="checkbox" name="hide" onchange="makeRequest()">Use Entry Form
</form>
</b>

</td>

<td width="29%" valign="top">
  <script language="JavaScript">
    document.write("<b>CURRENT SESSION NAME: ");
  </script>
  <b id="CurrentSessionName"></b>

<p>
<form>
  <select name="DropDownMenu" OnChange="changeSessionName(dropdownMenu.options[selectedIndex].value)">
    <option>Select a session name. . .
  </select>
  <script language="JavaScript">
    getSessionNames();
  </script>
</select>
</form>
</p>

<br>

<input type="button" value="Clear Markers"
  onclick="deleteMarkers()">
<input type="button" value="Delete Session"
  onclick="deleteSession()">

</td>
</table>

<input type="button" value="show cookie"
  onclick="showCookie()">

<br>

<script type="text/javascript">

```

```

function makeRequest()
    can be used
    {
        var mapStyleObject = document.getElementById("map").style;
        var locationObject = document.getElementById("location").style;
        var formObject = document.getElementById("form").style;
        if(document.getElementById.checked)
        {
            formObject.visibility = "visible";
            mapStyleObject.width = "700px";
            the right side of it
            locationObject.textAlign = "left";
            it is under the smaller map
        }
        else
        {
            formObject.visibility = "hidden";
            mapStyleObject.width = "1200px";
            locationObject.textAlign = "center";
        }
    }

function submitForm(name, address, phoneNumber, comment)
    {
        if (geocoder)
            created
            {
                geocoder.getLatLng(address, function(point)
                    {
                        if (!point)
                            alert(address + " not found");
                        else
                            //if the address can be found, begin assembling the info window's html
                            var commentString = "<html><head><style type='text/css'>";
                            //start of the html
                            if(comment != "")
                                //if no comment
                                is given, this CSS information is unnecessary
                                {
                                    commentString += "table.ridge{border-style: ridge}table.ridge{border-width: 5px}";
                                    //sets up the
                                    border for the comment
                                    commentString += "table.ridge{border-color: #0000ff}";
                                }
                            }
                    }
                )
            }
        //initializes the geocoder
    }
    //submits the contents of the entry form
    //if the geocoder has been
    //holds the maps style object
    //holds the current location display's style object
    //holds the entry form's style object
    //if the check box to use the entry form is checked
    //makes the entry form visible
    //makes the map's display smaller, thus hiding
    //moves the current location display to the left so
    //hides the entry form
    //makes the map full size
    //moves the current location display to the center

```

```

commentString += "b.gen{font-family: arial}</style></head>";
//sets up the font and boldness of the name, address and phone number
commentString += "<body><b class=\"gen\">"+name+"</b><br><b class=\"gen\">"+address+"</b><br>";
//adds the name, address and phone number to the body of the html document
commentString += "<b class=\"gen\">"+phoneNumber+"</b>";

if(comment != "")
    commentString += "<br><table class=\"ridge\"><tr><td> "+comment+" </tr></td></table>"; //if
a comment exists, add it to the pop up window

commentString += "</html>"; //closes the html document for the pop up window

createRecord(point.toString(), commentString) //calls the createRecord function to create a new
marker and record in the cookie
    }
    });
}
document.myForm.reset(); //resets the form
document.getElementById("colorDropDownMenu2").value = document.colorOptions.colorDropDownMenu.value;
//sets the value of the forms drop down menu to match the other color drop down menu
}

</script>

<!-- start of the form's html -->
<div id="form" style="font-weight:bold; font-
family:ARIAL;position:absolute;visibility:hidden;top:200px;left:800px;border-width:thick thick thick thick; border-
color:black; border-style:solid;">
<form name="myForm" action="#" onsubmit="submitForm(this.name.value, this.address.value, this.phoneNumber.value,
this.commentBox.value); return false">
<table cellpadding="10">

<tr><td><td><input name="name" type="text" size = "40"></td></tr>
<tr><td><td><input name="address" type="text" size = "40"></td></tr>
<tr><td><td><input name="phoneNumber" type="text" size = "40"></td></tr>

<tr><td>
<b>Marker Color:</b>
</td><td>
<select id="colorDropDownMenu2" OnChange="iconColor(colorDropDownMenu2.options[selectedIndex].value)">
<option value="red">Red
<option value="orange">Orange

```



```
<option value="brown">Brown
<option value="yellow">Yellow
<option value="green">Green
<option value="blue">Blue
<option value="purple">Purple
<option value="black">Black
<option value="gray">Gray
<option value="white">White
</select>
</td></tr>

<tr><td>
<b>Comments:</b>
</td><td>
<textarea name="commentBox" rows="5" cols="30"></textarea>
</td></tr>

<tr><td><input type="submit" value="Store a Marker" /></td></tr>
</table>
</form>
</div>

<!-- end of the form's html -->

<div id='map' style="width: 1200px; height: 600px"></div>

<b>
<div id='location' style = "text-align:center">CURRENT LOCATION:
<b id="message"></b></div>
</b>

</body>
</html>
```

Appendix B: Version 2 Code

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xmlns:v="urn:schemas-microsoft-com:vml">
<head>
<meta http-equiv="content-type" content="text/html; charset=utf-8"/>
<title>TJ Moore's Google Maps Gadget</title>

<script type="text/javascript">
```

```

    if (is_ie5up)
        document.write("<style type='text/css'> \v: * {behavior:url(#default#VML);}</style>");
</script>

<script
src="http://maps.google.com/maps?file=api&v=2&key=ABQIAAAA3jhyBvYFZJ15YJPUeKR6HRT2yXp_ZAY8_uFC3CFXnHIE1NvwkxQt3
n9NZNhrYXJNpZLGK5LNL2-N2Q"
type="text/javascript"></script>

<script type="text/javascript">
//<![CDATA[

    var map;
    var geocoder;
    var c_name;
    var icon;
    var color = "red";

    var centerlat;
    var centerlong;
    var centerZoom;

    var polyline = new Object();
    var currentMarkerID = 0;
    var checkForSessionValue;
    var stringObject = new Object();
    var httpReqObject = new Object();
    var mutex = "open";
    var idlookup = new Object();

    //this allows polylines to be shown in internet explorer
    //holds the polylines in object variables
    //holds the unique ID of the current marker
    //used to show whether proposed session name is currently in use
    var stringObject = new Object();
    var httpReqObject = new Object();
    var mutex = "open";
    var idlookup = new Object();

    //general variable for map object
    //used to store geocoder object
    //stores the c_name that corresponds to the user/session name given
    //variable used for custom icon
    //variable used for the icon's current color, default is red

    //variables used to hold location of map center

    function init()
    {
        //this function initializes items that only need to be called once
        if (GBrowserIsCompatible())
        {
            map = new GMap2(document.getElementById("map"));

            //map object, first created above

            geocoder = new GClientGeocoder();

            //geocoder object

            document.colorOptions.reset();
            document.geocode.reset();
            document.session.reset();
            document.makeCheckBox.reset();
            document.deleteMarkerCheckBox.reset();
            document.makeCheckBox.make.disabled = false;
            document.deleteMarkerCheckBox.disabled = false;

            //resets the radio buttons
            //resets the go to location field
            //resets the create new session field
            //resets the makeCheckBox checkbox
            //resets the deleteMarker checkbox
            //resets the makeCheckBox checkbox
            //enables the deleteMarker checkbox

```

```

document.makeCheckBox.comments.disabled = true;
document.changeCommentCheckBox.reset();
document.changeCommentCheckBox.changeComment.disabled = false;
document.makePolyline.reset();
document.makePolyline.makePoly.disabled = false;

setSessionName();

//This block initializes the icons to be used in the project
icon = new GIcon();
//makes a new icon object
icon.image = "http://labs.google.com/ridefinder/images/mm_20_red.png";
icon.shadow = "http://labs.google.com/ridefinder/images/mm_20_shadow.png";
icon.iconSize = new GSize(12, 20);
icon.shadowSize = new GSize(22, 20);
icon.iconAnchor = new GPoint(6, 20);
icon.infoWindowAnchor = new GPoint(5, 1);
load(); //calls the load function below
}

function load() //loads controls, holds the lat/long display listener, and calls the function to begin
displaying the stored data
{
    loadSessionCenter(); //loads the center location

    var center = map.getCenter();
    document.getElementById("message").innerHTML = center.toString(); //these two lines initialize the
lat/long display

    map.addControl(new GLargeMapControl());
    map.addControl(new GMapTypeControl());
    map.addControl(new GScaleControl());

    GEvent.addListener(map, "moveend", function()
    {
        var center = map.getCenter();
        document.getElementById("message").innerHTML = center.toString();
    });

    GEvent.addListener(map, "click", function(overlay, point)
    {
        clickListenerFunction(overlay, point);
    });
}
//calls function to test what click should do

```

```

document.getElementById("CurrentSessionName").innerHTML = c_name; //returns session name
parseString(); //begins using marker data
}

function makeRequest() //Sets up the HTTP request object based on the type of browser being used, tested for IE
and Firefox
{
    var info = new Date();
    var id = info.getTime(); //sets up a unique id, date in milliseconds, for use with the httpReqObject object
    to hold the unique httpReq objects

    if (window.XMLHttpRequest) //if it is firefox or another standard browser
    {
        httpReqObject[id] = new XMLHttpRequest();
    }
    else if (window.ActiveXObject) //if it is a version of Internet Explorer
    {
        httpReqObject[id] = new ActiveXObject("Microsoft.XMLHTTP");
    }

    if (!httpReqObject[id]) //if the object was not created
    {
        alert("Your browser does not support XMLHttpRequest technology!");
    }
    else
        return id; //returns the unique ID of the object to the caller
}

function setSessionName() //sets the current session name
{
    var id = makeRequest(); //calls for a new request object to be made

    if (httpReqObject[id] && mutex == "open") //if the object exists and the resource is open
    {
        idlookup["setSessionName"] = id; //stores the id for the finishing function
        mutex = "closed"; //closes the mutex
        try
        {
            httpReqObject[id].onreadystatechange = finishSetSessionName; //finishing function
            httpReqObject[id].open("GET", "setSessionName.php", true); //sets up the get request
            httpReqObject[id].setRequestHeader("Content-Type", "application/x-www-form-urlencoded"); //sets the
header

```

```

        httpReqObject[id].send(null);
    }
    catch(err)
    {
        alert(err);
    }
    mutex = "open";
    //opens the mutex
}

function finishSetSessionName()
{
    var id = idLookup["setSessionName"];
    //gets the appropriate request object id
    //alert("FSSN rs = "+httpReqObject[id].readyState+" status = "+httpReqObject[id].status+" mutex = "+mutex);
    if (httpReqObject[id].readyState == 4)
    {
        //if object is completely initialized
        if (httpReqObject[id].status == 200)
        {
            // only if "OK"
            {
                C_name = httpReqObject[id].responseText;
                //sets C_name to response sent from the php script
                //alert("CNAME = "+C_name );
            }
            else
            {
                alert("There was a problem retrieving the XML data:\n" + httpReqObject[id].statusText);
            }
        }
        if(C_name == 'undefined')
        {
            //if C_name is null, a C_name must be given by the
            user
            {
                C_name=prompt('Please enter your session name:', "");
                //prompts the user for a session/user name
            }
            if (C_name == null || C_name == "")
            {
                //if the C_name is null, set to 'default'
                C_name = "default";
            }
            storeNewSession(C_name);
            //calls function to store C_name in a database
            storeSessionCenter();
            //stores current location as the session center
        }
    }
}

function storeNewSession(tempCname)
{
    //stores new session name
    if(checkForSessionValue == 'found')
    {
        //if the C_name is already being used, don't overwrite
        var id = makeRequest();
    }
}

```

```

    if (httpReqObject[id] && mutex == "open")
    {
        mutex = "closed";
        try
        {
            //httpReqObject[id].onreadystatechange = finishStoreNewSession(id); //An event handler for
            //sets up the method
            httpReqObject[id].open("POST", "storeNewSession.php", true);
            httpReqObject[id].setRequestHeader("Content-type", "application/x-www-form-urlencoded"); //sets the
            header
            httpReqObject[id].setRequestHeader("Content-length", tempCname.length);
            httpReqObject[id].setRequestHeader("Connection", "close");
            //For future reference:
            //HTTP/1.1 defines the "close" connection option for the sender to
            //signal that the connection will be closed after completion of the response.
            httpReqObject[id].send(tempCname); //sends the data given
        }
        catch(err)
        {
            alert(err);
        }
        mutex = "open"; //opens the mutex
    }
    else
    {
        alert("Your browser does not support XMLHttpRequest technology!");
    }
    changeSessionName(tempCname); //calls function to set current session name database

}

function checkForSession(tempCname)
{
    //checks to see if the session name is already used
    var id = makeRequest(); //gets the id for the request object
    if (httpReqObject[id] && mutex == "open") //if the object exists and the mutex is open
    {
        idlookup["checkForSession"] = id; //store the id to be used by the finishing function
        mutex = "closed"; //close the mutex
        try
        {
            httpReqObject[id].onreadystatechange = finishcheckForSession; //sets up the finishing
            function
            httpReqObject[id].open("GET", "checkForSession.php"+"?currentSessionName="+c_name, true); //sets up
            the GET request, sends the current session name
        }
    }
}

```

```

    httpReqObject[id].setRequestHeader("Content-Type", "application/x-www-form-urlencoded"); //sets the
header information
    httpReqObject[id].send(null);
    }
    catch(err)
    {
        alert(err);
    }
    mutex = "open";

    //opens the mutex
}

function finishcheckForSession()
{
    var id = idlookup["checkForSession"]; //gets id from lookup object
    if (httpReqObject[id].readyState == 4) //if object is completely initialized
    {
        if (httpReqObject[id].status == 200) // only if "OK"
        {
            checkForSessionValue = httpReqObject[id].responseText; //receives information from
the php script
        }
        else
        {
            alert("There was a problem retrieving the XML data:\n" + httpReqObject[id].statusText);
        }
    }
}

function changesSessionName(tempCname)
{
    //stores the current session name
    var id = makeRequest(); //calls makeRequest to create the request object
    if (httpReqObject[id] && mutex == "open") //if the request object exists and the mutex is open
    {
        mutex = "closed"; //closes the mutex
        try
        {
            //httpReqObject[id].onreadystatechange = changesSessionName(id); //An event
            handler for an event that fires at every state change
            httpReqObject[id].open("POST", "changesSessionName.php", true);
            //sets up the method
            httpReqObject[id].setRequestHeader("Content-type", "application/x-www-form-urlencoded"); //sets the header
            httpReqObject[id].setRequestHeader("Content-length", tempCname.length);
            httpReqObject[id].setRequestHeader("Connection", "close");
            httpReqObject[id].send(tempCname);

            //sends the data given
        }
    }
}

```

```

    }
    catch(err)
    {
        alert(err);
    }
    mutex = "open";
}
//opens the mutex
else
{
    alert("Your browser does not support XMLHttpRequest technology!");
}
location.reload(true);
//reloads the browser so change takes effect
}
function storeSessionCenter()
{
    //stores the current center and zoom level
    getSessionCenter();
    //sets global center variables to current location
    var bold = document.createElement("B");
    //creates a bold text node to be used in the info
    window
    bold.appendChild(document.createTextNode("NEW MAP CENTER"));
    //Document Object Model (DOM) elements
    map.openInfoWindow(map.getCenter(), bold);
    //pop-up info box to show new center
    var poststr = "sessionName="+c_name+"&centerlat="+centerlat+"&centerlong="+centerlong+"&centerZoom="+centerZoom;
    //value to send in POST
    var id = makeRequest();
    //gets object ID
    if (httpReqObject[id] && mutex == "open")
    //if request object exists and the mutex is open
    {
        mutex = "closed";
        //closes the mutex
        try
        {
            //httpReqObject[id].onreadystatechange = finishstoreSessionCenter;
            //An event handler for an event that fires at every state change
            httpReqObject[id].open("POST", "storeSessionCenter.php", true);
            //sets up the method
            httpReqObject[id].setRequestHeader("Content-type", "application/x-www-form-urlencoded");
            //sets the header
            httpReqObject[id].setRequestHeader("Content-length", poststr.length);
            httpReqObject[id].send(poststr);
            //sends the data given
        }
        catch(err)
        {
            alert(err);
        }
    }
}

```



```

        mutex = "open";           //opens the mutex
    }
    {
        alert("Your browser does not support XMLHttpRequest technology!");
    }
}

function getSessionCenter()      //gets the current center and zoom level for the session
{
    var start, end;              //stores the start and end of the latitude and longitude
    var first, second;           //stores the latitude and longitude
    var center;                  //stores the center

    center = map.getCenter().toString();    //gets center and stores it as a string

    if(center != "null")           //if there is a center defined
    {
        start = center.indexOF(",") + 1;    //gets the start position of the latitude
        end = center.indexOF(",");          //gets the end position of the latitude
        first = center.substring(start,end); //stores the latitude

        start = center.indexOF(",") + 2;    //gets the start position of the longitude
        end = center.indexOF(",");          //gets the end position of the longitude
        second = center.substring(start,end); //stores the longitude

        centerLat = first;
        centerLong = second;
        centerZoom = map.getZoom();
    }

    //these three variables are global and keep track of center zoom and position

    function loadSessionCenter()      //loads the center and zoom level
    {
        var id = makeRequest();        //calls function to make the request

        if (httpReqObject[id] && mutex == "open")    //if the request exists and the mutex is open
        {
            idlookup["loadSessionCenter"] = id;    //stores the id for the finishing function
            mutex = "closed";
            try
            {
                httpReqObject[id].onreadystatechange = finishLoadSessionCenter;
            }
        }
    }

    finishing function                //sets the

```

```

        httpReqObject[id].open("GET", "loadSessionCenter.php?currentSessionName=" + c_name, true);
        //sets up the GET request and sends the current c_name
        httpReqObject[id].setRequestHeader("Content-Type", "application/x-www-form-urlencoded"); //sets the
header
    }
    httpReqObject[id].send(null);

    catch(err)
    {
        alert(err);
    }
    mutex = "open";

    //sets the mutex to open
}

function finishLoadSessionCenter()
{
    var id = idLookup["loadSessionCenter"];
    if (httpReqObject[id].readyState == 4) // only if req shows "loaded"
    {
        if (httpReqObject[id].status == 200) // only if "OK"
        {
            //alert(httpReqObject[id].responseText);
            eval(httpReqObject[id].responseText); //this should set centerLat, centerLong, and
centerZoom
        }
        map.setCenter(new GlatLng(centerLat, centerLong), centerZoom); //starting location and zoom level
    }
    else
    {
        alert("There was a problem retrieving the XML data:\n" + httpReqObject[id].statusText);
    }
}

function getSessionNames() //gets the session names
{
    var id = makeRequest(); //calls the function to make the request
    if (httpReqObject[id] && mutex == "open") //if the request object exists and the mutex is open
    {
        idLookup["getSessionNames"] = id; //stores the id in the lookup object
        mutex = "closed"; //closes the mutex
        try
        {

```

```

header
    httpReqObject[id].onreadystatechange = finishGetSessionNames;
    httpReqObject[id].open("GET", "getSessionNames.php", true);
    httpReqObject[id].setRequestHeader("Content-Type", "application/x-www-form-urlencoded"); //sets the
    httpReqObject[id].send(null);
}
catch(err)
{
    alert(err);
}
mutex = "open";
//set the mutex to open
}

function finishGetSessionNames()
{
    var id = idLookup["getSessionNames"];
    //gets the id from the lookup object
    if (httpReqObject[id].readyState == 4)
    {
        if (httpReqObject[id].status == 200)
        {
            // only if "OK"
            var cnames;
            //used to hold cnames
            eval(httpReqObject[id].responseText);
            //this should return the cnames so the HTML can be
            written
            for(x=0; x<i; x++)
            {
                document.write("<option value= \""+cnames[x]+">"+cnames[x]);
                //writes HTML for drop down menu
            }
        }
        else
        {
            alert("There was a problem retrieving the XML data:\n" + httpReqObject[id].statusText);
        }
    }
}

function deleteSession()
{
    //this deletes all session information
    var id = makeRequest();
    //makes the request object

```

```

    if (httpReqObject[id] && mutex == "open")           //if the request object exists and the mutex is open
    {
        mutex = "closed";                               //closes the mutex
        try
        {
            //httpReqObject[id].onreadystatechange = finishDeleteSession;
            handler for an event that fires at every state change
            httpReqObject[id].open("POST", "deleteSession.php", true);
            //sets up the method
            httpReqObject[id].setRequestHeader("Content-type", "application/x-www-form-urlencoded"); //sets the header
            httpReqObject[id].setRequestHeader("Content-Length", name.length);
            httpReqObject[id].setRequestHeader("Connection", "close");
            httpReqObject[id].send(c_name);              //sends the data given
        }
        catch(err)
        {
            alert(err);
        }
        mutex = "open";                                //opens the mutex
    }
    else
    {
        alert("Your browser does not support XMLHttpRequest technology!");
    }
    location.reload(true);                             //reloads the browser so change takes effect
}
//PARSESTRING AND USECOOKIES WERE COMBINED

function parseString()                                //gets the stored information from the database
{
    var id = makeRequest();                             //makes the request object
    if (httpReqObject[id] && mutex == "open")           //if the object exists and the mutex is open
    {
        idLookup["parseString"] = id;                  //stores object id into object
        mutex = "closed";                               //closes the mutex
        try
        {
            httpReqObject[id].onreadystatechange = finishParseString; //sets up the finishing
        }
        httpReqObject[id].open("GET", "parseString.php" + '?currentSessionName=' + c_name, true); //sets up
        the GET request
        httpReqObject[id].setRequestHeader("Content-Type", "application/x-www-form-urlencoded"); //sets the
        HTTP header

```

```

        httpReqObject[id].send(null);
    }
    catch (err)
    {
        alert(err);
    }
    mutex = "open";
}

//opens the mutex

function finishParseString()
{
    var id = idlookup["parseString"];
    if (httpReqObject[id].readyState == 4) //only if req shows "loaded"
    {
        if (httpReqObject[id].status == 200) //only if "OK"
        {
            var queryNum;
            var string;

            //should be set by the php string

            eval(httpReqObject[id].responseText); //should return marker information

            for (var i=0; i<queryNum; i+=5)
            {
                //marker data is grouped in fives
                //the first data
                element is the marker ID, given by date in milliseconds
                coord = new GlatLng(string[i+1], string[i+2]); //coordinates are the second and third data
                elements
                color = string[i+4]; //retrieves the color stored in the
                database
                icon.image = "http://labs.google.com/ridefinder/images/mm_20_"+color+".png"; //sets the color of the
                icon to match the saved color
                createMarker(coord, string[i+3], string[i]); //this calls the function to create a marker
            }
            color = "red"; //red is the default color
            icon.image = "http://labs.google.com/ridefinder/images/mm_20_"+color+".png"; //sets color back to default
        }
        getPolyLines(); //calls the function to get the polyLines from the database
    }
    else
    {
        alert("There was a problem retrieving the XML data:\n" + httpReqObject[id].statusText);
    }
}

```

```

    }

    function getPolyLines()           //gets the polyline data from the database
    {
        var id = makeRequest();        //makes the request and returns its id
        if (httpReqObject[id] && mutex == "open") //if the object exists and the mutex is open
        {
            mutex = "closed";          //closes the mutex
            try
            {
                idlookup["getPolyLines"] = id; //stores the request object id
                httpReqObject[id].onreadystatechange = finishGetPolyLines; //sets up the finishing function
                httpReqObject[id].open("GET", "getPolyLines.php" + '?currentSessionName=' + c_name, true); //sets up
                the GET request
                httpReqObject[id].setRequestHeader("Content-Type", "application/x-www-form-urlencoded"); //sets up
                the header
                httpReqObject[id].send(null);

            }
            catch(err)
            {
                alert(err);
            }
            mutex = "open"; //opens the mutex
        }
    }

    function finishGetPolyLines()
    {
        var id = idlookup["getPolyLines"];

        if (httpReqObject[id].readyState == 4) // only if req shows "loaded"
        {
            if (httpReqObject[id].status == 200) // only if "OK"
            {
                var polylineNum; //gives the number of polylines returned
                var string; //holds the unique ids used to define a polyline

                eval(httpReqObject[id].responseText); //stores the polyline information in string and the
                number returned in polylineNum

                for(var i=0; i<polylineNum; i+=2) //polylineNum gives number of polylines returned
                {
                    currentMarkerID = string[i]; //gets first ID, sets to currentMarkerID
                }
            }
        }
    }
}

```

```

        makeNewPoly(string[i+1], "false"); //gets second ID, calls function
    }
    else
    {
        alert("There was a problem retrieving the XML data:\n" + httpReqObject[id].statusText);
    }
}

function clickListenerFunction(marker, point) //used by mouse click listeners to determine what action is to be
taken
{
    if(marker) //if there is a marker at the given
        location
        {
            if(document.deleteMarkerCheckBox.deleteMarker.checked) //if the delete marker checkbox is checked
            {
                map.removeOverlay(marker); //remove the overlay from the map
                deleteOneMarker(marker.info, "true"); //delete the particular marker info from the
                database
            }
            else if(document.changeCommentCheckBox.changeComment.checked)
            {
                changeComments(marker.info); //changes marker comments
                map.removeOverlay(marker); //remove the overlay from the map
            }
            else if(document.makePolyLine.makePoly.checked)
            {
                makeNewPoly(marker.info, "true"); //accepts marker ID
            }
        }
        else
        {
            if(document.makePolyLine.makePoly.checked) //if makeCheckBox is checked, but marker was
            not clicked //make a record at the click location
            {
                var newMarker = createRecord(point.toString()); //with a polyline attached
                makeNewPoly(newMarker.info, "true");
            }
            else if(document.makeCheckBox.make.checked) //if makeCheckBox is checked
            createRecord(point.toString()); //make a record at the click location
        }
    }
}

```

```

    }
}

function deleteOneMarker(info, deletePoly) //deletes the information of the particular marker
{
    var id = makeRequest();
    if (req)
    {
        try
        {
            //httpReqObject[id].onreadystatechange = finishDeleteOneMarker(id);
            handler for an event that fires at every state change
            httpReqObject[id].open("POST", "deleteOneMarker.php", true);
            httpReqObject[id].setRequestHeader("Content-type", "application/x-www-form-urlencoded"); //sets the method
            httpReqObject[id].setRequestHeader("Content-length", name.length); //sets the header
            httpReqObject[id].setRequestHeader("Connection", "close");
            httpReqObject[id].send("markerID="+info+"&database="+c_name); //sends the data given

            if(deletePoly == "true")
                deletePolylines(info); //deletes the polyline associated with this marker
        }
        catch(err)
        {
            alert(err);
        }
    }
    else
    {
        alert("Your browser does not support XMLHttpRequest technology!");
    }
}

function deletePolylines(info) //deletes the polylines from the database
{
    var id = makeRequest();
    if (httpReqObject[id] && mutex == "open")
    {
        mutex = "closed";
        try
        {
            //httpReqObject[id].onreadystatechange = finishDeletePolylines(id);
            handler for an event that fires at every state change
        }
    }
}

```



```

        httpReqObject[id].open("POST", "deletePolyLines.php", true);
        //sets up the method
        httpReqObject[id].setRequestHeader("Content-type", "application/x-www-form-urlencoded"); //sets the header
        httpReqObject[id].setRequestHeader("Content-length", name.length);
        httpReqObject[id].setRequestHeader("Connection", "close");
        httpReqObject[id].send("polyID="+info+"&sessionName="+C_name);
        //sends the data given, which is the session_name in this case
    }
    catch(err)
    {
        alert(err);
    }
    mutex = "open"; //opens the mutex
}
else
{
    alert("Your browser does not support XMLHttpRequest technology!");
}
}

function changeComments(markerID) //function to change the comments of existing markers
{
    var string = {}; //holds properties from database

    string = getMarkerInfo(markerID); //gets the markers information from the database

    lat =string[0]; //stores marker property values into respective variables
    long = string[1];
    comment = string[2];
    tempColor = string[3];

    deleteOneMarker(markerID, "false"); //calls the function to delete the marker, don't remove polyline

    comment = prompt('Please enter a new comment:', ""); //prompts the user for a new comment to be added
    if(comment == null) //if the comment is null, don't show any value
        comment = "";

    icon.image = "http://labs.google.com/ridefinder/images/mm_20_"+tempColor+".png"; //sets the icon global
    variable temporarily

    var id = makeRequest();
    if (httpReqObject[id] && mutex == "open")
    {
        mutex = "closed";
    }
}

```

```

    try
    {
        //An event
        handler for an event that fires at every state change
        httpReqObject[id].open("POST", "changeComments.php", true);
        //sets up the method
        httpReqObject[id].setRequestHeader("Content-type", "application/x-www-form-urlencoded"); //sets the header
        httpReqObject[id].setRequestHeader("Content-length", name.length);
        httpReqObject[id].setRequestHeader("Connection", "close");
        httpReqObject[id].send("polyID="+info+"&sessionName="+c_name+"&comment="+comment); //sends the data given,
        //which is the session_name in this case
    }
    catch(err)
    {
        alert(err);
    }
    else
    {
        mutex = "open"; //sets the mutex to open
    }
    else
    {
        alert("Your browser does not support XMLHttpRequest technology!");
    }

    coord = new GLatlng(lat, long); //makes the new coordinate object

    createMarker(coord, comment, markerID); //calls the create marker function

    icon.image = "http://labs.google.com/ridefinder/images/mm_20_"+color+".png"; //the icon is set back to the
    original color
}

function getMarkerInfo(markerID) //gets the marker information from the database
{
    var id = makeRequest(); //makes the request object

    if (httpReqObject[id] && mutex == "open") //if the request object exists and the mutex is open
    {
        idlookup["getMarkerInfo"] = id; //store id in the lookup table
        mutex = "closed"; //close the mutex
        try
        {
            httpReqObject[id].onreadystatechange = finishGetMarkerInfo;
            httpReqObject[id].open("GET", "getMarkerInfo.php"?currentSessionName="+c_name +
            "&markerID="+markerID, true); //set up GET request
        }
    }
}

```

```

header
    httpReqObject[id].setRequestHeader("Content-Type", "application/x-www-form-urlencoded"); //set
    httpReqObject[id].send(null);
}

catch(err)
{
    alert(err);
}
mutex = "open"; //open the mutex
}

function finishGetMarkerInfo()
{
    var id = idLookup("getMarkerInfo");
    if (httpReqObject[id].readyState == 4) // only if req shows "loaded"
    {
        if (httpReqObject[id].status == 200) // only if "OK"
        {
            eval(httpReqObject[id].responseText);
        }
    }
    //runs javascript code written in php

    //the first data element is the marker ID, given by date in milliseconds
    coord = new GLatLng(stringObject[markerID][i+1], stringObject[markerID][i+2]); //coordinates are the
    second and third data elements
    color = string[markerID][i+4];
    //retrieves the color stored in the database
    icon.image = "http://labs.google.com/ridefinder/images/mm_20_"+color+".png"; //sets the color of the icon
    to match the saved color
    createMarker(coord, string[markerID][i+3], string[markerID][i]); //this calls the
    function to create a marker
    color = "red";
    //red is the default color
    icon.image = "http://labs.google.com/ridefinder/images/mm_20_"+color+".png"; //sets color back to default
    getPolyLines();
}
else
{
    alert("There was a problem retrieving the XML data:\n" + httpReqObject[id].statusText);
}
}

function createRecord(location) //creates a record from the lat and long given by click
{

```

```

var start, end; //stores the start and end of the latitude and longitude
var first, second; //stores the latitude and longitude
var coord; //stores the lat/long object google maps uses
var info = ""; //stores comment info

var marker; //stores marker object

if(document.makeCheckBox.comments.checked)
{
    info=prompt('Enter marker comment:', ""); //prompts the user for a marker comment
    if(info == null)
        info = "";
}

start = location.indexOf("(") + 1; //gets the start position of the latitude
end = location.indexOf(","); //gets the end position of the latitude
first = location.substring(start,end); //stores the latitude

start = location.indexOf(",") + 2; //gets the start position of the longitude
end = location.indexOf(")",start); //gets the end position of the longitude

second = location.substring(start,end); //stores the longitude

coord = new GLatlng(first, second); //makes a coordinate object
marker = createMarker(coord,info); //creates a marker

var id = makeRequest(); //calls the function to make the request and
//stores the id //if the request object exists and the mutex is open
if (httpReqObject[id] && mutex == "open")
{
    mutex = "closed"; //closes the mutex
    try
    {
        //httpReqObject[id].onreadystatechange = finishCreateRecord(id);
        //An event handler for an event that fires at every state change
        httpReqObject[id].open("POST", "createRecord.php", true);
        httpReqObject[id].setRequestHeader("Content-type", "application/x-www-form-urlencoded"); //sets up the POST method
        httpReqObject[id].setRequestHeader("Content-length", name.length); //sets the header
        httpReqObject[id].setRequestHeader("Connection", "close");
    }
    httpReqObject[id].send("sessionName="+c_name+"&markerID="+marker.info+"&lat="+first+"&long="+second+"&info="+info+"&col
or="+color); //sends the marker data given
}
catch(err)
{
    alert(err);
}

```

```

    }
    mutex = "open";           //opens the mutex
  }
  else
  {
    alert("Your browser does not support XMLHttpRequest technology!");
  }

  return marker;              //returns the marker object to the calling function
}

function createMarker(point, message, info)           //creates markers with lat/long and marker's message and unique
  identifier
{
  var marker = new GMarker(point, icon);              //new marker
  if(info == "")                                     //if there is no previous information
  {
    info = new Date();                                //date in milliseconds, used as a unique identifier
    marker.info = info.getTime();
  }
  else
  {
    marker.info = info;                               //sets marker info from the info from the
    database
  }

  map.addOverlay(marker);                               //adds marker to map

  GEvent.addListener(marker, "click", function()      //calls click listener function
  {
    markerClickListener(marker, message);              //calls markerClickListener function
  });
  return marker;                                       //returns the marker
}

function markerClickListener(marker, message)          //function used to create the marker's listener
{
  //as long as left click is not being used for any other purpose, show pop up window
  if(!document.deleteMarkerCheckbox.deleteMarker.checked &&
    !document.changeCommentCheckbox.changeComment.checked &&
    !document.makePolyLine.makePoly.checked)
  {
    marker.openInfoWindowHtml("<big>" + message + "</big>");
  }
}

function deleteMarkers()                               //this clears all markers from the map
{

```

```

map.clearOverlays(); //this clears the markers without having to reload the
browser with location.reload(true);

var id = makeRequest(); //calls the function to make the request object and stores
the id
{
    if (httpReqObject[id] && mutex == "open") //if the request object exists and the mutex is open
    {
        mutex = "closed"; //closes the mutex
        try
        {
            //httpReqObject[id].onreadystatechange = finishDeleteMarkers(id);
            //An event handler for an event that fires at every state change
            httpReqObject[id].open("POST", "deleteMarkers.php", true);
            httpReqObject[id].setRequestHeader("Content-type", "application/x-www-form-urlencoded"); //sets up the method
            httpReqObject[id].setRequestHeader("Content-length", name.length); //sets the header
            httpReqObject[id].setRequestHeader("Connection", "close");
            httpReqObject[id].send("sessionName="+c_name); //sends the data
        }
        catch(err)
        {
            alert(err);
        }
        mutex = "open"; //opens the mutex
    }
    else
    {
        alert("Your browser does not support XMLHttpRequest technology!");
    }
}

function makeNewPoly(markerID, newMarker) //makes a new polyline between two markers
{
    if(currentMarkerID == 0) //a line must have two points
    {
        currentMarkerID = markerID; //currentMarkerID is a global variable that stores the first
        marker's ID
    }
    else if(currentMarkerID == markerID)
    {
        //do nothing ;
    }
    else
    {
        polyline
        {
            //do nothing if the same marker is clicked for the
        }
    }
}

```

```

var markerID1 = currentMarkerID;           //ID of first marker
var markerID2 = markerID;                  //ID of second marker

var string1 = [];                          //string array to hold first markers properties
var string2 = [];                          //string array to hold second markers properties
var points = [];                           //holds polyline's points
var lineColor;                             //holds color of first marker to decide what color to make the
polyline

var red = "#ff0000";                       //colors for polylines
var blue = "#0000ff";
var black = "#000000";
var green = "#00ff00";
var brown = "#a37500";
var white = "#ffffff";
var yellow = "#ffff00";
var orange = "#eebb00";
var gray = "#aaaaaa";
var purple = "#bf00be";

string1 = getMarkerInfo(markerID1);        //gets first marker's info
string2 = getMarkerInfo(markerID2);        //gets second marker's info

points[0] = new GLatLng(string1[0], string1[1]); //string#[0] == latitude
points[1] = new GLatLng(string2[0], string2[1]); //string#[1] == longitude

lineColor = string1[3];                    //color to use for polyline

var mstring1;                              //used to concatenate the two IDs into one integer
var mstring2;

mstring1 = markerID1.toString();           //convert ints to strings
mstring2 = markerID2.toString();

if(newMarker == "true")                   //if this is a new marker, create object and store in the
{
    storePoly(mstring1, mstring2);
}

switch(lineColor)
{
    case "red": polyline[mstring1+mstring2] = new GPolyline(points, red,3,0.7); //creates
    stores in an array/dictionary
    break;
}

```

```

        case "blue": polyline[mstring1+mstring2] = new GPolyline(points, blue,3,0.7);
        stores in an array/dictionary
        break;
        case "black": polyline[mstring1+mstring2] = new GPolyline(points, black,3,0.7);
        stores in an array/dictionary
        break;
        case "green": polyline[mstring1+mstring2] = new GPolyline(points, green,3,0.7);
        stores in an array/dictionary
        break;
        case "brown": polyline[mstring1+mstring2] = new GPolyline(points, brown,3,0.7);
        stores in an array/dictionary
        break;
        case "white": polyline[mstring1+mstring2] = new GPolyline(points, white,3,0.7);
        stores in an array/dictionary
        break;
        case "yellow": polyline[mstring1+mstring2] = new GPolyline(points, yellow,3,0.7);
        stores in an array/dictionary
        break;
        case "orange": polyline[mstring1+mstring2] = new GPolyline(points, orange,3,0.7);
        stores in an array/dictionary
        break;
        case "gray": polyline[mstring1+mstring2] = new GPolyline(points, gray,3,0.7);
        stores in an array/dictionary
        break;
        case "purple": polyline[mstring1+mstring2] = new GPolyline(points, purple,3,0.7);
        stores in an array/dictionary
        break;
    }

    map.addOverlay(polyline[mstring1+mstring2]); //adds polyline to the map

    currentMarkerID = 0; //resets currentMarkerID

}

function storePoly(ID1, ID2) //stores the polyline in the database
{
    var id = makeRequest(); //makes the request object and stores the id
    if (httpReqObject[id] && mutex == "open") //if the request object exists and the mutex is open
    {
        mutex = "closed"; //closes the mutex
        try
        {

```



```

//httpReqObject[id].onreadystatechange = finishStorePoly(id);
//\n event
handler for an event that fires at every state change
    httpReqObject[id].open("POST", "storePoly.php", true);
//sets up the method
    httpReqObject[id].setRequestHeader("Content-type", "application/x-www-form-urlencoded"); //sets the header
    httpReqObject[id].setRequestHeader("Content-length", name.length);
    httpReqObject[id].setRequestHeader("Connection", "close");
    httpReqObject[id].send("sessionName="+c_name+"&ID1="+ID1+"&ID2="+ID2);
//sends the data given
}
    catch(err)
    {
        alert(err);
    }
    else
    {
        mutex = "open";
        //opens the mutex
    }
    else
    {
        alert("Your browser does not support XMLHttpRequest technology!");
    }
}

function disableComment()
{
    //disables makeComment checkbox if makeCheckBox is not checked
}

if(document.makeCheckBox.checked)
{
    //if the makeComment checkbox
    is checked:
    {
        document.makeCheckBox.comments.disabled = false;
        document.deleteMarkerCheckBox.deleteMarker.disabled = true;
        document.deleteMarkerCheckBox.reset();
        //enables the comment checkbox
        //disables delete marker checkbox
        //resets delete marker

checkbox
        document.changeCommentCheckBox.changeComment.disabled = true;
        document.makePolyLine.makePoly.disabled = true;
        //disables the change comment checkbox
        //disables the make polyline

checkbox
    }
    else if(document.deleteMarkerCheckBox.deleteMarker.checked)
    {
        //if the delete marker checkbox is
        checked:
        {
            document.makeCheckBox.make.disabled = true;
            document.changeCommentCheckBox.changeComment.disabled = true;
            document.makePolyLine.makePoly.disabled = true;
            //disables the make comment
            //disables the change comment checkbox
            //disables the make polyline

checkbox
        }
    }
}

```

```

        else if(document.changeCommentCheckBox.checked) //if the change comment checkbox is
        { //disables the make comment
            document.makeCheckBox.make.disabled = true;
        }
checkbox
document.deleteMarkerCheckBox.deleteMarker.disabled = true; //disables delete marker checkbox
document.makePolyLine.makePoly.disabled = true; //disables the make polyline
checkbox
}
        else if(document.makePolyLine.makePoly.checked) //if the make polyline checkbox is
        { //disables the make comment
            document.makeCheckBox.make.disabled = true; //disables the change comment checkbox
checkbox
            document.changeCommentCheckBox.changeComment.disabled = true; //disables delete marker checkbox
            document.deleteMarkerCheckBox.deleteMarker.disabled = true; //disables delete marker checkbox
        }
        else //if none of the above checkboxes are checked:
        { //disables the comment checkbox
            document.makeCheckBox.comments.disabled = true; //resets the
            document.makeCheckBox.reset();
        } //enables the deleteMarkerCheckBox
commentCheckBox checkbox to unchecked //enables the make comment checkbox
checkbox
document.deleteMarkerCheckBox.deleteMarker.disabled = false; //enables the change comment checkbox
checkbox
document.makeCheckBox.make.disabled = false; //enables the make polyline
checkbox
document.changeCommentCheckBox.changeComment.disabled = false;
document.makePolyLine.makePoly.disabled = false;
checkbox
}
}
function iconColor(changedColor) //changes the color of the icons used by the program
{
    color = changedColor;
    //changes current color to the value selected
    icon.image = "http://labs.google.com/ridefinder/images/mm_20_"+color+".png"; //calls the right icon from
    google's server
}
function showAddress(address) //function for the geocode search bar displayed on top
{

```

```

        if (geocoder)
        {
            geocoder.getLatLng(address, function(point) {
                if (!point)
                    alert(address + " not found");
                else
                    map.setCenter(point, 13);
            });
        }
    }

    //]]>
</script>

</head>
<body onload="init()" onunload="GUnload()">

<table border="0" width="100%" cellpadding="0">

<td width="43%" valign="top">

    <form name="geocode" action="#" onsubmit="showAddress(this.address.value); return false">
        <input type="text" size="60" name="address" value="" />
        <input type="submit" value="Go to New Location" />
        <!--<b>jumps to a location.</b>-->
    </form>

</td>

<p>
<form name="session" action="#" onsubmit="storeNewSession(this.sessionName.value); return false">
    <input type="text" size="60" name="sessionName" value="" />
    <input type="submit" value="Create New Session" />
    <!--<b>Creates a new session.</b>-->
</form>
</p>

<br>

<form name = "colorOptions">

<input type="button" value="Store Session Center"
    onclick="storeSessionCenter()">
<b>&nbsp;&nbsp; Store center and zoom level.&nbsp;&nbsp;&nbsp;</b></b>

<select name="colorDropDownMenu" OnChange="iconColor(colorDropDownMenu.options[selectedIndex].value)">

```

```

<option value="red">Red
<option value="orange">Orange
<option value="brown">Brown
<option value="yellow">Yellow
<option value="green">Green
<option value="blue">Blue
<option value="purple">Purple
<option value="black">Black
<option value="gray">Gray
<option value="white">White
</select>
<b>&nbsp;Marker Color</b>

</form>

</td>

<td width="26%" valign="top">
<b>
  <form name="makeCheckBox">
    <input type="checkbox" name="make" onchange="disableComment()">Make a marker on click
    <input type="checkbox" name="comments">with a comment.
  </form>
</b>

<b>
  <form name="changeCommentCheckBox">
    <input type="checkbox" name="changeComment" onchange="disableComment()">Change an existing marker's comment.
  </form>
</b>

<b>
  <form name="deleteMarkerCheckBox">
    <input type="checkbox" name="deleteMarker" onchange="disableComment()">Delete a marker when it is clicked.
  </form>
</b>

<br>

<b>
  <form name="makePolyline">
    <input type="checkbox" name="makePoly" onchange="disableComment()">Make a polyline between two markers.
  </form>
</b>

</td>

```

```
<td width="29%" valign="top">
<script language="JavaScript">
document.write("<b>CURRENT SESSION NAME: ");
</script>
<b id="CurrentSessionName"></b>

<p>
<form>
<select name="dropDownMenu" onChange="changeSessionName(dropDownMenu.options[selectedIndex].value)">
<option>Select a session name. . .
<script language="JavaScript">
getSessionNames();
</script>
</select>
</form>
</p>

<br>

<input type="button" value="Clear Markers"
onClick="deleteMarkers()">
<input type="button" value="Delete Session"
onClick="deleteSession()">

</td>
</table>

<br>

<div id="map" style="width:1200px; height: 600px"></div>
<b>
<div ALIGN = "center">CURRENT LOCATION:
<b id="message"></b></div>
</b>
</body>
</html>
```

Appendix C: PHP Functions

Group 1: Initialization

setSessionName.php

```
<?php
$server = 'localhost';           //stores the mysql connection information
$username = 'root';
$password = 'topsecretps';
$database = 'googlemapsproject';
```

```
$mysqli = mysqli_connect($server, $username, $password, $database);           //connects to mysql

$sql = "SELECT * FROM current_c_name_that_is_being_used_by_this_program";           //stores the sql query
$res = mysqli_query($mysqli, $sql);           //runs the mysql query
$numRows = mysqli_num_rows($res);           //stores the number of rows received

if($numRows <= 0)           //if there is no information to return, return undefined
{
    echo "undefined";
}
else
{
    $row = mysqli_fetch_array($res, MYSQL_BOTH);           //store the result and echo it back to the javascript
    code           echo $row[0];
}

mysqli_close($mysqli);           //close the mysql connection
?>
```

loadSessionCenter.php

```
<?PHP
$sessionName = $_GET['currentSessionName'];           //stores variable from GET
$server = 'localhost';           //stores mysql connection information
```

```

$username = 'root';
$password = 'topsecretps';
$database = 'googlemapsproject';

$mysqli = mysqli_connect($server, $username, $password, $database); //connects to mysql

$sql = "SELECT * FROM sessions WHERE sessionName='". $sessionName."'"; //store sql query

$res = mysqli_query($mysqli, $sql); //runs sql query

$numRows = mysqli_num_rows($res); //stores the number of rows returned

if($numRows > 0) //if there were rows returned
{
    $row = mysqli_fetch_array($res, MYSQL_BOTH); //stores returned data in an array

    echo "centerlat = ".$row[0]."; centerlong = ".$row[1]."; centerZoom = ".$row[2].";" //sets the location variables in javascript
}
else
{
    echo "centerlat = 21.94304553343818; centerlong = 11.6015625; centerZoom = 2;" //sets the default location variables in javascript
    //default location and zoom level allows the majority of the world to be seen
}
mysqli_close($mysqli); //closes mysql connection
?>

```

getSessionNames.php

```

<?PHP

$server = 'localhost'; //stores connection information
$username = 'root';
$password = 'topsecretps';
$database = 'googlemapsproject';

$mysqli = mysqli_connect($server, $username, $password, $database); //connects to mysql

$sql = "SELECT sessionName FROM sessions"; //stores the query

$res = mysqli_query($mysqli, $sql); //runs the sql query

```

```

$row = mysqli_fetch_array($res, MYSQL_BOTH); //gets array returned from query
$numRows = mysqli_num_rows($res); //stores the number of rows returned
if($row > 0) //if the number of rows is greater than zero, initialize the cnames array
    echo "cnames = []";
for($j=0; $j<$numRows; $j++) //for each row, store the session names in the array
{
    echo "cnames[".$j."] = ".$row[$j].",";
}
mysqli_close($mysqli); //close the mysql connection
?>

```

parseString.php

```

<?PHP
$stable = $_GET['currentSessionName']; //takes the number from URL
$server = 'localhost'; //stores connection information
$username = 'root';
$password = 'topsecretps';
$database = 'googlemapsproject';
$mysqli = mysqli_connect($server, $username, $password, $database); //sets up mysql connection
$sql = "SELECT * FROM ".$stable."_markers"; //stores sql query
$res = mysqli_query($mysqli, $sql); //runs sql query
$numRows = mysqli_num_rows($res); //stores the number of rows returned
echo "queryNum = ".$numRows; //sets the queryNum variable on the javascript side
echo "string = new Array("; //sets up string array
for($i=0; $row = mysqli_fetch_array($res, MYSQL_BOTH); $i++) //stores marker info in the array
{
    for($j=0; $j<5; $j++)
    {
        echo $row[$j].", "; //this will result in an extra comma, and may cause a problem
    }
}

```



```

    }

    echo " );";

    mysqli_close($mysqli);           //closes the mysql connection
?>

```

getPolyLines.php

```

<?PHP
$table = $_GET['currentSessionName']; //takes the number from URL

$server = 'localhost';           //stores connection information
$username = 'root';
$password = 'topsecretpts';
$database = 'googlemapsproject';

$mysqli = mysqli_connect($server, $username, $password, $database); //connects to mysql

$sql = "SELECT * FROM ".$table."_polylines";           //stores sql query

$res = mysqli_query($mysqli, $sql);           //runs sql query

$numRows = mysqli_num_rows($res);           //stores the number of rows returned

echo "polylineNum = ".$numRows;           //sets the number of polylines

echo "string = new Array(";           //initializes a new array

for($i=0; $row = mysqli_fetch_array($res, MYSQL_BOTH); $i++) //stores the new polyline information in the array
{
    for($j=0; $j<2; $j++)
    {
        echo $row[$j].", ";           //this will result in an extra comma, and may cause a problem
    }
}

echo " );";

mysqli_close($mysqli);           //closes the mysql connection
?>

```

Group II: Creation

storeNewSession.php

```

<?php
$sessionName = $_POST;           //stores data from POST

$server = 'localhost';           //stores connection information
$username = 'root';
$password = 'topsecretps';
$database = 'googlemapsproject';

$mysqli = mysqli_connect($server, $username, $password, $database); //connects to mysql

$sql = "INSERT INTO current_c_name_that_is_being_used_by_this_program (currentSessionName) VALUES ($_POST)";
//stores sql query

$res = mysqli_query($mysqli, $sql); //runs sql query

$sql = "INSERT INTO sessions (sessionName) VALUES ($_POST)"; //stores sql query

$res = mysqli_query($mysqli, $sql); //runs sql query

$sql = "CREATE TABLE ".$sessionName."_markers ( markerID CHAR(13) PRIMARY KEY, lat TEXT, lng TEXT, comment TEXT, color
TEXT, ) ";

marker table for the new session name //stores sql to create a

$res = mysqli_query($mysqli, $sql); //runs sql to create table

$sql = "CREATE TABLE ".$sessionName."_polylines ( markerID1 CHAR(13), markerID2 CHAR(13))"; //stores sql to create
a polyline table for the new session name

$res = mysqli_query($mysqli, $sql); //runs sql

mysqli_close($mysqli); //closes mysql connection
}
?>

```

checkForSession.php

```

<?PHP
$server = 'localhost';           //stores the connection information
$username = 'root';
$password = 'topsecretps';
$database = 'googlemapsproject';

$mysqli = mysqli_connect($server, $username, $password, $database);    //connects to mysql

$sql = "EXPLAIN session";           //stores sql query
$res = mysqli_query($mysqli, $sql);    //runs sql query

$row = mysqli_fetch_array($res, MYSQL_BOTH);    //stores the number of rows returned
$numRows = mysqli_num_rows($res);

$flag = 0;           //if 1, name found, if 0, name not found

for($i=0; $i<$numRows; $i++)    //if new session name is the same as one that is already in the database, it cannot
    be used
    if($row[$i] == $_GET
        $flag = 1;

    if($flag == 1)
        echo "found";
    else
        echo "not found";

mysqli_close($mysqli);           //closes mysql connection
?>

storePoly.php

<?PHP
$sessionName = $_POST['sessionName']+"_polylines";           //store the variables received from the POST
$ID1 = $_POST['ID1'];

```

```

$ID2 = $_POST['ID2'];

$server = 'localhost';           //stores the mysql connection information
$username = 'root';
$password = 'topsecretps';
$database = 'googlemapsproject';

$mysqli = mysqli_connect($server, $username, $password, $database);           //connects to mysql
$sql = "INSERT INTO ".$sessionName." VALUES ('".$ID1."', '".$ID2."')";           //stores sql query
$res = mysqli_query($mysqli, $sql);           //runs sql query

mysqli_close($mysqli);           //closes mysql connection
?>

```

createRecord.php

```

<?PHP
$sessionName = $_POST['sessionName']."_markers";           //stores data sent by POST
$markerID = $_POST['markerID'];
$lat = $_POST['lat'];
$long = $_POST['long'];
$comment = $_POST['info'];
$color = $_POST['color'];

$server = 'localhost';           //stores the connection information
$username = 'root';
$password = 'topsecretps';
$database = 'googlemapsproject';

$mysqli = mysqli_connect($server, $username, $password, $database);           //connects to mysql
$sql = "INSERT INTO ".$sessionName." VALUES ('".$markerID."', '".$lat."', '".$long."', '".$comment."', '".$color."')";
//stores sql query

$res = mysqli_query($mysqli, $sql);           //runs sql query

mysqli_close($mysqli);           //closes mysql connection
?>

```

storeSessionCenter.php

```

<?PHP
$sessionName = $_POST['sessionName'];           //gets variables from POST data
$centerLat = $_POST['centerLat'];
$centerLong = $_POST['centerLong'];
$centerZoom = $_POST['centerZoom'];

$server = 'localhost';                           //stores the server connection information
$username = 'root';
$password = 'topsecretps';
$database = 'googlemapsproject';

$mysqli = mysqli_connect($server, $username, $password, $database); //sets up the server connection

$sql = "INSERT INTO sessions VALUES ('".$sessionName."', '".$centerLat."', ".$centerLong."', ".$centerZoom.");"
//stores the sql query

$res = mysqli_query($mysqli, $sql);               //runs the sql query

mysqli_close($mysqli);                           //closes the mysql connection
?>

```

Group III: Revision*changeSessionName.php*

```

<?php
$mysqli = mysqli_connect("localhost", "root", "topsecretps", "googlemapsproject"); //sets up the mysql
connection

if(mysqli_connect_errno())                        //if there are no errors
{
    echo "Connect failed: " .mysqli_connect_error()."\n";
    exit();
}
else
{
    $sql = "DELETE FROM Current_C_name_that_is_being_used_by_this_program"; //stores the sql query

    $res = mysqli_query($mysqli, $sql);         //runs the
    sql query
}

```

```

        $sql = "INSERT INTO current_c_name_that_is_being_used_by_this_program VALUES ('".$_POST."");" //stores
the sql query

        $res = mysqli_query($mysqli, $sql);
        //runs the sql query

        if($res == TRUE) //displays results
        {
            echo "A record has been inserted";
        }
        else
        {
            echo "Could not insert record: ".mysqli_error($mysqli);
        }

        mysqli_close($mysqli); //closes the connection
    }
}
?>

```

changeComments.php

```

<?PHP
$markerID = $_POST['polyID']; //gets the variables from POST
$sessionName = $_POST['sessionName']+ "_polyLines";
$comment = $_POST['comment'];

$server = 'localhost'; //stores the mysql connection information
$username = 'root';
$password = 'topsecrets';
$database = 'googlemapsproject';

mysqli = mysqli_connect($server, $username, $password, $database); //connects to mysql

$sql = "UPDATE ".$sessionName." SET comment = '".$_comment."' WHERE markerID = '".$_markerIDstuart."'"; //stores
the sql query

$res = mysqli_query($mysqli, $sql); //run the sql query

mysqli_close($mysqli); //close the mysql connection
?>

```

Group IV: Deletion

deleteSession.php

```
<?PHP
$sessionName = $_POST;           //stores data sent by POST

$server = 'localhost';           //stores connection information
$username = 'root';
$password = 'topsecretps';
$database = 'googlemapsproject';

$mysqli = mysqli_connect($server, $username, $password, $database); //connects to mysql

$sql = "DELETE ".$_POST." FROM current_C_name_that_is_being_used_by_this_program"; //stores sql query

mysqli_query($mysqli, $sql);      //runs sql query

$sql = "DROP TABLE ".$_POST."_markers"; //stores sql query

mysqli_query($mysqli, $sql);      //runs sql query

$sql = "DROP TABLE ".$_POST."_polylines"; //stores sql query

mysqli_query($mysqli, $sql);      //runs sql query

mysqli_close($mysqli);           //closes mysql connection
?>
```

deleteOneMarker.php

```
<?PHP
$markerID = $_POST['markerID']; //stores data returned by POST
$sessionName = $_POST['database']."_markers";

$server = 'localhost';           //stores connection information
$username = 'root';
$password = 'topsecretps';
$database = 'googlemapsproject';
```

```

$mysqli = mysqli_connect($server, $username, $password, $database);           //connects to mysql
$sql = "DELETE FROM ".$sessionName." WHERE markerID='".$markerID.'"";           //stores sql query
$res = mysqli_query($mysqli, $sql);           //runs sql query
mysqli_close($mysqli);           //closes mysql connection
?>

```

deletePolylines.php

```

<?PHP
$polyID = $_POST['polyID'];           //stores data received by POST
$sessionName = $_POST['sessionName'].$_polyID;

$server = 'localhost';           //stores mysql connection information
$username = 'root';
$password = 'topsecretps';
$database = 'googlemapsproject';

$mysqli = mysqli_connect($server, $username, $password, $database);           //connects to mysql

$sql = "DELETE FROM ".$sessionName." WHERE (markerID='".$polyID.'" OR markerID2='".$polyID.'"");           //stores
sql query

$res = mysqli_query($mysqli, $sql);           //runs sql query

mysqli_close($mysqli);           //closes sql connection
?>

```

deleteMarkers.php

```

<?PHP
$sessionName = $_POST['database'].$_markers";           //gets data returned from POST

$server = 'localhost';           //stores connection information
$username = 'root';
$password = 'topsecretps';
$database = 'googlemapsproject';

$mysqli = mysqli_connect($server, $username, $password, $database);           //connects to mysql

$sql = "DELETE FROM ".$sessionName;           //stores sql query

```


getMarkerInfo.php

```
<?PHP
$stable = $_GET['currentSessionName']; //takes the number from URL
$markerID = $_GET['markerID']; //gets the markerID

$server = 'localhost'; //stores mysql connection information
$username = 'root';
$password = 'topsecretips';
$database = 'googlemapsproject';

$mysqli = mysqli_connect($server, $username, $password, $database); //connects to mysql

$sql = "SELECT * FROM ".$stable."_markers WHERE markerID='".$markerID."'"; //stores sql query

$res = mysqli_query($mysqli, $sql); //runs sql query

$row = mysqli_fetch_array($res, MYSQL_BOTH); //stores number of rows returned

for($j=0; $j<5; $j++) //stores information in an array
{
    echo "responseObject[".$markerID."][$j."."] = ".$row[$j].".";
}

mysqli_close($mysqli); //closes mysql connection
?>
```

Appendix D: Project Log

6/15/06:

Started worked with the google maps API. Signed up for an API key, but then realized that, for practice purposes, I didn't need one. For API documentation, see <http://www.google.com/apis/maps/> and for the API key, see https://www.google.com/accounts/Login?continue=http://maps.google.com/maps/api_signup%3Furl%3Dhttps%253A%252F%252Fcipc.navy.mil&followup=http://maps.google.com/maps/api_signup%3Furl%3Dhttps%253A%252F%252Fcipc.navy.mil (need to log in to google account to get access). I will continue working with the google api next week. I will also need to go over some different web technologies, especially XHTML and Javascript, before really doing anything that interesting.

6/19/06:

Continued working with the Google Maps API. I was able to merge some of the examples given in the documentation with my own project so that it now gives a latitude/longitude location. I can also add points to the map with information when the point is clicked.

6/21/06:

I began going through the Javascript tutorial at www.w3schools.com. I stopped at http://www.w3schools.com/js/js_break.asp.

6/22/06:

Continued working with Javascript and the Google Maps API. I completed merging all of the API Documentation examples that I picked out together, and I am fairly pleased with the outcome. Now that I have a better understanding of the API, I would like to be able to store data in cookies so that a user would be able to save the work they accomplish. I began doing this with a sort of "log on" that takes in a person's name, but this would be the same for anyone using the computer. The next main step is to store data points in the cookies, then maybe see if a true user environment could be set up. I made an effort to add more comments to the code so that I will be able to have a better idea what I was doing when I continue my work on Monday.

6/26/06 & 6/27/06:

I was really focused on this project, so I decided to combine two days worth of worklogs together. I started working on storing cookies that contain lat/long information. One thing I discovered is that while Internet Explorer stores cookies separately, Firefox merges them into one text document. This probably does not affect the code, however. Beginning with the basic cookie functions I found at www.w3schools.com, I was able to begin to start modifying the functions to store the data I would need. Some major problems were encountered when I started to try dealing with the lat/long data and how to best store it. In the general Google API form, the coordinates are stored as (coordinate, coordinate), but when trying to make the markers with the data in this form, it doesn't seem to convert properly, so I changed it so each lat/long is stored separately in the form (data)(data)(data). . . I can now add markers and comments, but there are some hiccups

when adding the second marker (don't know why the second one causes the problem. . .) and when the page is closed and reopened, the data is lost, although refreshing does not lose the data. Overall, I have made a great deal of progress and have learned a lot about javascript, the google maps api, and cookie storage.

6/28/06:

Finished working on Google Maps API project and the rough draft of the documentation. See documentation for more details.

6/29/06:

Finished the final draft of my Google Maps API project's documentation and forwarded it to Mike for review.

7/10/06:

Discussed project direction with several people in CIPC. Worked with Google Maps API to try and develop multiple users' cookies. The function worked perfectly to hold the cookies, but the delete markers function will not work with the new adaptation. I will continue to work with this problem and see if I can find a suitable delete function for the project.

7/11/06:

Found problem with delete cookie function in project. After the `c_name`, there needed to be an "=" sign added to the string. Seems a bit temperamental to me, but that small difference kept the function from working correctly. Now, when the program is loaded, it prompts the user for a session name, then that is used as the `c_name` to store the marker information given by the user. Next time the page is loaded, it will again prompt for the session name. If the same session name is given, the markers will be loaded from last time. If a new session name is a different session name is given, any markers for that name will be loaded. The delete markers button will clear the cookie for the current session.

I also began looking into how to make a drop down menu that stores the various session names used by the program. I made a good bit of progress towards that end. First, a function had to be made to parse through the cookie and pick out all the `c_names/sessionnames/usernames` found in the cookie. Then, I had to learn how to make a drop down menu that not only held the information, but also wrote the drop down items on its own whenever the user makes a change. The program is not working perfectly, but I do have a drop down menu that contains the session names and a function that is getting close to being able to use the drop down menu to switch to different sessions. I will keep working with it tomorrow, and I think I should be able to make more progress in this area. Overall, my progress was pretty good today, and I am pleased with where the project is at the moment.

7/12/06:

I made a good bit of progress regarding the drop down menu and how the program adds new session names. The code will need to be cleaned up, and I am reloading the page more than I'd like to, but I think the bulk of this step in the project is done. Tomorrow I

will continue working with it and try to make it function more smoothly. Also, after talking with Jay, he suggested trying to eliminate as many page reloads as possible, possibly by using AJAX. I will most likely talk with him again tomorrow and see how that might be incorporated into my project. Things that need to be corrected is that a new session name should be added as soon as the name is created, not just after a marker is stored. Also, after a session is deleted, the program should switch to a different user.

7/13/06:

The functionality I was working on adding to the project in this phase is basically complete. Besides the initial API controls, among which a map scale has been added, the program can now add new session names through the use of a text box and button. The current session is displayed, and all current sessions are listed in a drop down menu. The user also now has the option to either clear the markers or to completely delete the session. The overall layout of the project has been revamped with better organized HTML that put the information in an invisible table. Left to do for this phase of the project is to see how to update the items on the screen without refreshing it, possibly changing how the user chooses the session name to use after deleting a session, and finishing the documentation. There may be several other things that could be accomplished in this part of the project, but that is all I can think of right now.

7/17/06:

Looked at several websites to try and find a good example of a Javascript implementation of a right mouse click control. I didn't really find any good examples, so I will have to keep looking. My idea is to alter the way markers are stored from being a text box and button method to requiring the user to right click in a location, then bring up a prompt box for the comment. I am not sure if this will work, since the Google maps API does not seem to support right-clicking, but I will at least give it a shot.

7/18/06:

Continued looking for a good right click Javascript example. I found one that was pretty good, but I'm not sure how I can incorporate it into my current project, or if I will be able to have the right click work with the google maps coordinate system.

7/19/06:

Continued to look for a solution to the right click problem for the Google Maps API project. One interesting idea proposed by Jay was to make a toggle switch so that both functions could be performed by the left mouse button. This way, the functionality could be added without having to figure out how to get the right mouse button to work.

7/20/06:

Continued working towards a solution for the Google Maps project. Instead of the right click idea mentioned previously, I began looking into some form of a toggle switch for the left mouse button so that, depending on the option selected, it could either be used to place markers or solely to navigate. This seemed to be a complicated problem, however, since once a listener is turned on, I was not able to determine how to turn it off. So, I will most likely look at that more on Monday. Another thing that I discovered is a website

that shows what events are called when different selections are made in google maps. From this, I discovered that the right mouse button does seem to register, but only as a mousedown event. This may prove to be a promising solution to the problem, but I will have to wait and see if I can use it in the implementation.

7/24/06:

Continued working with the Google Maps API project. To take a break from the "click to add marker" problem, I began researching how to make markers of different colors. After poking around online with the API, I found a suitable function and determined what colors are available. From this, I think it would be nice if I could add maybe some radio buttons to change which color markers to make. I think I will see if I can just use the left button functionality to add markers to the project, although this method may make it difficult to capture the lat/long of the point on the map. I will have to look into this issue and see if it will pose a problem.

7/25/06:

Continued working with the Google Maps API project. I changed the code to add a marker on left click, then added an optional pop-up box to store a comment for the marker. Once I did this, I realized that I could use a similar method for the code to add a marker, allowing me to have a checkbox for adding a marker as well. This was possible by not imbedding the function to add a marker into the listener, where, once added, it could not respond to changes in a flag variable. Instead, I just had the listener function call another function that held the actual functionality, where I am then able to modify the flag variables. I also added radio buttons that allow for changes in the marker color, and I save this choice in a fourth parentheses set, so now each marker has four data items instead of three. The main things left to do is to add a way to change the starting location for the user and to clean up the user interface and the code, as well as write new documentation.

7/26/06:

Revamped the user interface to the project and ensured that, after restructuring the HTML, everything still functioned properly. I worked on how to reset the base location, but found it to be more complicated than I originally thought. I also worked on graying out the comment marker check box, but had some difficulty with that also. Overall, the project is taking shape pretty well, but there are still a few major things left to do.

7/27/06:

I reached a good stopping point in the project. Currently, the marker creation, session creation, and other commands are working correctly. I managed to create a function to handle the disabling of the comment checkbox, and I added comments to most of the new functionality, so the project is fairly solid at this point. I began to work on the controls to allow for the changing of the map's center, but this will be quite complicated because of the current cookie structure. I did, however, get a good start on it, so I can continue working with that next week. I will be updating the documentation to reflect the new changes, so that should explain all of the functionality that has been added lately.

7/31/06:

Added the ability to choose a new starting location to the google maps project. This was somewhat more complicated than some of the previous steps, since it required changes to the cookie structure. It is working well now, with the current structure as follows:

```
[center lat][center long][zoom level]null(marker lat)(marker long)(comment)(color). . .
```

Some things left to do:

- add some sort of visual indicator for when the center location is changed
- clean up the user interface, as needed
- clean up the code, adding comments where needed
- rewrite the documentation to reflect the new changes

Once all of this is done, I believe it may warrant a '2.0' designation.

8/1/06:

After considering several options, I added a pop up comment box to indicate when a new center location is chosen. Before continuing with the other steps, I am trying to decide whether to add a separate control for the map view so that it can be saved as part of the user session, as well as looking into a way to delete a marker when clicking on it. Once I decide what to do about these two issues, then I will continue with the previously listed things to do.

8/3/06:

Started working on how to add delete on click functionality to the markers. It was very frustrating, but I think I at least have a better idea of what the problem entails. The click listeners I had set up need to be combined somehow, but at least I have the cookie structure under control. I will have to work more with it next week.

8/7/06:

Tried to restructure some of my javascript functions in order to fix my current problem with the google maps listeners, but I still have not found a way for the delete and info window listeners to work together. I downloaded a listener example from the google maps api page so I can began experimenting with a way to get the listeners to cooperate. I will hopefully be able to straighten it out tomorrow. I talked with Dr. Briner on Friday, and he had a few suggestions for my project:

- more clearly define the problem
- incorporate a database, most probably a xml database
- clean up the code, use a developer's tool to restructure the comments and indentations, ie use something like visual studio .net

8/8/06:

Continued working with the google maps project. I found out that the reason my delete functions were not working properly is because I was creating multiple layers of markers at once, thereby causing the delete functions to not work correctly. I have fixed the few

instances of this problem that I found, and *I believe* the majority of the bugs are worked out. I do, however, continue to find things here and there, such as problems when reloading the page with the dynamic check boxes being disabling accidentally, which have been fixed. Basically, what needs to be done to accomplish a 2.0 version level, the code must be cleaned up, new documentation created, and beta testing must be continued. After that is accomplished, I will move into researching how to incorporate an XML database into the project.

8/9/06:

Continued debugging google maps project. Found that there was a problem with the click to delete function where the wrong marker would be deleted from the cookie. The problem was that the unique id stored in the cookie was not being loaded and a new id was being used. This problem has been fixed. More problems are likely still present.

8/10/06:

Continued working with Google Maps project. Continued testing and restructured the code so that it follows a more logical order and grouped functions that work together. Also did a good bit of testing to find any problems that may exist in the program, seems to be functioning pretty well. (Might have forgotten something, it is currently Monday.)

8/14/06:

Continued researching how javascript and xml can work together to form a database. I found and bookmarked several good websites dealing with the subject and believe I am on my way to finding a solution. Progress was slow going, but I have some information worth looking at. Also, there may need to be some differences between how Internet Explorer and Firefox will work with XML, but at least one of the websites I found deals with the issue.

8/15/06:

Continued looking at how to implement an XML database. Since Javascript, and most likely a Java applet, is not easily capable of outputting data to a file, another solution will need to be found. Jay suggested creating a simple server to handle the data I/O, which sounds like a good idea to me. I looked at some different possibilities, and I have decided that a Java based server is probably the way to go. I will then get to review Java while I continue onto the next step of my project.

8/16/06:

Continued looking at what to do with the google project and how to go about making a xml server.

8/17/06:

Continued looking up how to implement a Java server. I have some code to work with now, so I need to start customizing it to fit my requirements.

8/25/06:

Fixed a problem with html pop up window.

8/28/06:

Worked with the Google maps project and looked at different implementations of polylines. I finally got a simple polyline display to work, and I am considering different options as to how I could incorporate them into my project. One idea is to allow the lines to be put anywhere, but this may make it hard to get rid of the polylines. Another idea is to tie the polylines to markers, such that a marker would serve as a vertex for the lines, but this may pose problems for how to remove the lines as well as what to do if a marker is removed.

8/29/06:

Looked at how to implement polylines in google maps. Matt suggested allowing for distance calculations and making a graph structure to find shortest distance between points. It looks like I may have to use markers as vertexes to allow for the removal of polylines, but this creates a good bit of problems that will have to be looked at.

9/1/06:

Continued looking at the google maps polyline issue.

9/5/06:

Removing individual overlays can be done by assigning the particular polyline to a variable which can then be removed later (see example of dynamic polylines for more information). I started getting the polyline idea to work. I can now add polylines between markers with two clicks. I have to figure out how to store the information, but I am fairly pleased with my progress so far. Note how the polyline's unique ID is created, by using a number concatenated from the two markers' unique IDs.

9/6/06:

Continued working on google maps project. Trying to get polylines to work properly. Talked to Jay and decided a separate function would be the best way to define the colors for the polyline and allow for blending of the colors between the two markers. I still have to develop the functions to save the polyline data to the cookie, but I think the best way to do it is by adding another section to the cookie for polylines and use the markers' unique IDs to define the lines. The polyline goal should be the last thing I do with the project before moving to the server implementation portion.

9/11/06:

Continued working with Google Maps project. Decided to simply let the polyline identifiers 'float' within the other marker information, which leads to a more loose and simple structure than some of the structures I was considering before. I still have to have the information read, processed, and then deleted, but this represents a solid move in the right direction.

9/12/06:

Worked on how to delete stored polyline information. Algorithm should be as follows:
Go through cookie and find matching ID to markerID of deleted marker

Once one is found, note whether it is first or second in the statement so the whole thing can be properly deleted
Delete the whole statement
Save the cookie
Go through the entire process again until all instances of the marker's ID are gone

9/13/06:

Continued working on Google Maps project. I am getting closer to finishing the polyline functions, but I am having some problems with the backwards search function I wrote. The functions are working great for one polyline, but with more than one, problems sometimes come up. The most serious is that at time, the wrong line is removed, which makes me think that the function may be overlooking the proper function and skipping to a different one. I will hopefully be able to correct this problem today.

9/14/06:

Finished up polyline storage in the cookies. Had a few issues, such as having to put a control statement in the change comment function to keep from removing the polylines, as well as various problems related to string parsing. The biggest problem that had to be solved was that the array that held the polylines could not hold the concatenated markerID index values, since the maximum integer value is somewhere around 9,000,000,000,000,000. As per Jay's recommendation, I changed the polyline storage from an array to an Object, which could then be indexed by a string, which allowed for proper polyline marker's to be stored.

9/18/06:

Revised Google maps code. Removed unescapes throughout, most of the getElementById calls, and put a control to keep users from making a polyline from a marker back to itself. Started looking at AJAX, see Tim Hibbard's site, as per Dr. Briner.

9/19/06:

Started looking at how to implement a Java server. I will most likely set up a server that takes in the parameters from javascript and puts them into a XML document. This server should also be able to handle information returns as well. Although I am not positive as of right now, I may be writing most of the new code for the server mostly on the server side with java with less new code on the javascript side.

9/20/06:

Continued looking at a Java server implementation.

9/21/06:

Continued working with my Java server and client. I am getting closer to getting to the bare essential elements of the server and client and should be able to start rebuilding them into what I need them to do.

9/25/06:

Continued working on Java server and client. The server and client are now functioning well, so the next step is to decide how to implement the google maps xml database with them. NOTE: I've probably read this before, but you cannot compare two strings with '==', you must use the compare string function.

9/26/06:

Continued looking at servers, and found that the Google API has some inherent XML parsing. However, I don't know if it allows for rewriting the document.

9/27/06:

Continued working on Java server and how to work with XML in Java. There are several libraries in the JavaDoc that deal with xml manipulation, but I'm not sure if they are capable of doing what I need them to do. Also, yesterday I came up with a pretty decent idea for sending the data from client to server in a fairly simple matter. Each data element will be sent as a string with a number to indicate its type, such that my cookie structure would have the following idea when being sent:

[0][1][2]DELIMITER(3)(4)(5)(6)(7){8,9}

However, this would not account for session names. . . More on this to come.

9/29/06:

Continued looking at how to work with XML. Also came up with a XML schema for the project that I think should work.

```
<cname cname="default">
<startLoc>
<lat>53.1254</lat>
<lng>-80.1234</lng>
<zoom>12</zoom>
</startLoc>
```

```
<displayData>
  <marker uniqueID="12345" lat="43.65654" lng="-79.90138" info="Some stuff to
display in the<br>First Info Window" color="red"/>
  <marker uniqueID="32456" lat="43.91892" lng="-78.89231" info="Some stuff to
display in the<br>Second Info Window" color="blue"/>
  <polyline id1="12345" id2="32456"/>
  <marker uniqueID="45678" lat="43.82589" lng="-79.10040" info="Some stuff to
display in the<br>Third Info Window" color="green"/>
</displayData>
</cname>
```

```
<cname cname ="Current_C_name_that_is_being_used_by_this_program">
default
</cname>
```

10/2/06:

Continued looking at how to implement an XML server with Java, but I continue to have trouble finding good information about how to do it. I am starting to consider a simpler alternative that would basically use a simple text file with a Java server. I'm not really sure which way to go, and I am beginning to get somewhat frustrated.

10/3/06:

Continued looking at Java XML implementation. It is beginning to look like the best way to do this is to use the javax.xml parsers along with my own create and update controls. More research is still necessary.

10/4/06:

Google web kit / dynamic user interface meeting at 8:00am. After that, I spent some time continuing my server research. Two websites to look at:

<http://developer.yahoo.com/yui/>

<http://code.google.com/webtoolkit/>

10/5/06:

Started working on xml document creator. It needs some work, but it seems to have potential, although it might be better if I used some of the java api's built in functionality. I'll have to keep working on it.

10/6/06:

Kept working on java xml creator. The recursion method I'm using needs a little bit of tweaking, but I think it is pretty close. I still need to figure out what functionality the java api offers for xml CRUD operations.

10/10/06:

Continued working on Google Maps server idea. I am not sure if the XML server plan will work, so I may just try to get a server working while keeping my existing data structure from the cookie. I talked with Dr. Briner about this briefly, and he said that this idea may be a good starting point, then I could convert it to xml. So, my new focus will be on getting a Javascript application to communicate with my Java server.

10/11/06:

Kept looking at AJAX javascript web connections and I also looked at the Yahoo user interface to see about using their toolkit to set up the connection, which would allow for better browser independence. I will continue looking at that today.

10/12/06:

Looking at AJAX and server stuff, Jay recommended learning more about the apache server and using that to handle the javascript http connection. We came up with the following to do list:

- 1) LEARN APACHE

- 2) Serve page from Apache instead of locally
- 3) Set up database with table or tables for storing map data
- 4) Write a script in PHP/Ruby that takes user input and writes it to a database (also read from database)
- 5) Configure Apache to run PHP scripts
- 6) Change original HTML to php and write/read to/from database instead of cookie
- 7) . . .
- 8) Profit

10/13/06:

Kept looking at Google Maps stuff. I think I will need to get a book on some of this stuff to figure out how to set my project up.

10/16/06:

Started working with the book I bought about PHP, MySQL and Apache. Started setting some of the stuff up, but I will need to more carefully read the book to get a good idea of what I need to do. I also modified Apache's index so that it links to some other simple HTML pages.

10/17/06:

Kept looking at Apache server, php, and mysql. Also installed Visual Studio .NET on my computer along with the MSDN. I also wrote a small C++ program to test that everything was working properly.

10/18/06:

Looked at some .NET stuff. Still need to get a handle on what needs to be done with Apache, PHP, and MySQL.

10/19/06:

Kept looking at Apache stuff. Talked with Matt and discovered the Vertrigo Server that comes with Apache, MySQL, PHP and more already installed and properly configured, so I think I will try to work with that instead. Talked to Dr. Briner about what to do, he told me to look into CGI scripts to work with the HTTP connections. He also, once again, pressed for something more on my project. One idea I thought of today is to make a geography game where you have to choose the correct capital. I have to get the database and server stuff to work before I can really get into that though.

10/20/06:

Started trying to learn about PHP scripting. Went over a couple of chapters in the book I purchased.

10/23/06:

Looked at some PHP stuff. I think I am going to put a little more focus right now on setting up a simple HTTP client/server interaction. I have set the following timeline for my senior project:

Project Goals and Timeline:

By Oct 27th:

I plan to have a simple HTTP post/get examples working with client and server interaction.

By Nov. 10th:

I plan to have a working PHP script to handle client data sent to the server and write it to the database.

By Nov. 17th:

I plan to have working MySQL queries to handle CRUD operations.

By Nov. 24th:

I plan to implement usability examples.

By Dec. 1st:

I plan to finish my projects documentation.

10/24/06:

Kept working with PHP and AJAX to figure out how to set up HTTP client/server interactions. I have a couple of examples that look promising.

10/25/06:

Looked at some PHP stuff and kept working on how to use a HTTP connection.

10/26/06:

Kept working with PHP, javascript, and server stuff.

10/27/06:

Worked with MySQL. I made a separate database and a new table, then I modified a php script to insert a record into the table. The main issue I need to work on at the moment is the ability to communicate between javascript and php. If I can figure that out, I should be in pretty good shape to move on to my next goal.

10/30/06:

Continued working on Google Maps project. I found an example of javascript and php interacting in one program which I think I might be able to use to have my javascript talk with my database through php. I hope to have the project working by the end of this week or early next week.

10/31/06:

I kept looking for a good way for php and javascript to talk to each other. I found a few examples of javascript HTTP connections and other scripting examples online, but I am still not sure I have the right solution yet.

11/1/06:

Kept working with Javascript/AJAX stuff to try and set up an HTTP connection between my javascript and the php server side stuff.

11/2/06:

I finally found some decent examples of GET and POST in Javascript. The GET example is especially interesting because it works with a php script that runs a query against a MySQL database. I feel like I am getting very close to what I need to get done, but there still remains a good bit of work before I can set this database up to work the same way as the cookie is working now.

11/3/06:

Kept working with the Javascript GET and POST. I now have a decent example of both GET and POST to modify for my project. Some main things that I learned are how PHP handles the GET and POST requests, such as how the string is parsed. For instance, if sending a string, the first variable will be recognized as a variable, but subsequent variable declarations must be delimited by an ampersand, ie
`var1=value1&var2=value2&var3=value3. . .`

11/6/06:

Worked on some project documentation for the class meeting today. I also looked up the structure of an HTTP request header for reference to what needs to be set when using the XMLHttpRequest object in javascript.

11/7/06:

Worked on some networking stuff. Had the integration meeting today to discuss what was going on with various projects. There will be a CMMI review by SPAWAR on November 30th.

11/8/06:

GET and POST uses:

Use GET to retrieve information to the server.

Use POST to alter the data in the server.

Worked on understanding POST request better and began looking into how to set up SQL queries to work with Google Maps.

11/9/06:

Continued working on Google Maps project. Trying to figure out how to set up the POST and GET HTTPRequests to work between PHP and Javascript.

11/13/06:

Focused on the POST request between PHP and Javascript. I am trying to send an array from PHP to Javascript, and I am not quite sure how to get this to work. I think that I should send a php string that can be interpreted by Javascript as an array. That way, I can use the data that is sent however I choose.

11/14/06:

Hurray for the eval() function! From PHP, I can send a string that instantiates an array, then, once it is received by Javascript, I call the eval() function, which executes the string. That way, I can easily send a string array from PHP to Javascript.

Study: What functions does the cookie perform that the database must perform?

- 1) Pulls all current markers and polylines for session name to map on startup
- 2) Stores all current session names and marker and polyline info
- 3) Stores session name currently in use
- 4) Creates, reads, updates, and deletes marker and polyline information
- 5) Stores Session start location and zoom level

11/15/06:

Finally found a way to return multiple query lines to PHP from MySQL. I discovered that by using a loop with the `mysqli_fetch_array()` to retrieve the information, I can get all of the information I need from a given table. I also learned the difference from the `mysql_*` family of functions and the `mysqli_*` family of functions. The `mysqli_*` family should be used with all newer versions of mysql. Explained more in the book.

11/16/06:

Since I have cobbled together all of the pieces that I needed to start the conversion from cookie storage to database storage. As I began, I realized that it would require many http requests to perform all of the necessary functions. I am attempting to get everything in place, then I may try to consolidate the http connections. A great deal of the load of this method will be placed on the server side php script, so, altogether, this may take longer than I was hoping, since debugging could be difficult with all of the integrated technologies. I stopped at the 'useCookies()' function for today, I should hopefully be able to at least get the javascript converted tomorrow, but there will still remain a lot of php stuff to do. Hopefully, it will go quickly. I also have to make sure that php is sending the proper stuff back to the javascript, especially since

11/17/06:

Continued working on my conversion from cookie storage to database storage. I think it will take a good bit more work to do the conversion than I originally thought, especially since I have to write a good bunch of PHP scripts to handle all of the CRUD operations. In any case, I will continue to work on this as much as I am able, since it needs to all be finished up fairly soon.

11/20/06:

Continued working on Google maps project conversion and PHP script creation. I realized that I cannot return a value from the onreadystatechange function, so I either have to combine the functions together in a different way then before (see parseString function) or I have to define a global variable to store the value (see checkForSession function). I hope to finish this conversion as soon as possible.

11/21/06:

Continued working to convert my project over to working with the MySQL database. Same as usual, just setting up a bunch of HTTP connections and PHP scripts.

11/22/06:

Continued working to convert my project. . .

Thanksgiving "break" log:

11/23/06:

Finished up a good bit of the program conversion, but there is still a lot of debugging to do.

11/24/06:

Had a major problem with connecting to php scripts because I was not enclosing the urls in paranthesis to send them as strings, so my php scripts were not being reached, and my code was not working at all. This was extremely frustrating.

11/25/06:

Sharing the request objects seems to be causing some problems, I am going to try and set up a way so that there will be separate request objects for each http request, maybe this will solve some of my problems. I also discovered that on my POST request functions, I misnamed the postReq 'req' in the if statement controlling access to the function, thereby killing all of my POST functionality :D I read online that some errors can be caused by httprequest objects overwriting each other, so I set it up so that each httprequest is unique, but I still am having the same problem of the ready state staying at 0, meaning it is uninitialized. I am pretty confused at this point as to how I can fix this problem.

End of Thanksgiving log

11/27/06:

Kept working on Google maps project. I discovered that the .onreadystatechange function cannot pass a value to the finishing function, so I cannot pass my ids to my functions as I was trying to do. Instead, they need to be global variables so that the finishing function can access the ids.

11/28/06:

Looked at connex container flow chart to show how cargo entering the country makes its way by land, sea, and air. Walked through the flow chart with Matt, Mike and Jay. After meeting with Dr. Briner yesterday, it has been decided that I should start looking at how to prepare my project paper, documentation, and defense instead of spending that time on getting my project to work. Although I would still like to finish that portion of the project, it will no longer be my number one priority. I began looking into creating a comment entering form for my Google Maps project without much luck. I will continue looking into how that can be done.

11/29/06:

Worked on usability example for my Google Maps project. I discovered that I do not need to use AJAX to have dynamic forms, but instead should be using DHTML. Basically, I will set some of the forms to invisible, then make the data entry forms visible, which will allow me to write the formatted data to the same cookie.

11/30/06:

Continued working on the form for the google maps project. I set up the dhtml so that the form would show up when the map's width is toggled to be smaller. I got the name, address, and phone number to be correctly stored, as well as passing the address to the geocoder for processing. In the evening, I talked with Dr. Briner about what else might need done. Dr. Briner suggested adding a comment box and also showing the marker color info in the form as well. I will finish work on getting that stuff in the form tomorrow.

12/1/06:

I had a little trouble setting up all the form stuff properly, but I finally got most of it to work together. A big problem I had was how to get the two marker color drop down menus to update each other correctly, but after a little bit of tweaking on the getelementbyid object, I got it to work. Another problem occurred when I started using HTML with CSS to style the information being put in the marker info window. The main problem was the CSS used a semicolon as a delimiter, which could not be stored in the cookie, so I rewrote the CSS so it didn't need the semicolon, and now it is working pretty well, although the internet connection has been pretty lousy today.